

NNAMDI AZIKIWE UNIVERSITY, AWKA

Faculty of Physical Sciences
Department of Computer Science

SKYROUTE

A Flight Booking Simulation System

Designed and implemented with Next.js, React and TypeScript, using the browser's own localStorage as the sole persistence mechanism

TECHNICAL REPORT

Submitted in partial fulfilment of the requirements for the course

CSC 422 — Software Engineering

SUBMITTED BY

A GROUP OF NINETEEN STUDENTS

(Full membership listed overleaf)

SUPERVISED BY

Dr. Alade

August, 2026

Group Members

The project was carried out by the nineteen students listed below. Implementation of the software was undertaken by a single developer; the remaining members were responsible for requirements, documentation, testing and review. Section 6.4 sets out how the group was organised and what each sub-team was responsible for.

S/N	Name	Matriculation Number	Sub-team	Principal Role
1	Ezedimbu Ebube Emmanuel	2022514081	Development	Team Lead / Sole Developer
2	Uba Thelma Chidinma	2022514029	Coordination & Administration	Assistant Coordinator / Secretary
3	Nnolum Vivian Chisom	2022514008	Coordination & Administration	Scheduling & Meeting Records Officer
4	Micheal Ezra Chukwuebuka	2022514105	Coordination & Administration	Resources & Logistics Officer
5	Anuforo Valentine Uzochukwu	2022514073	Requirements & Research	Requirements Gathering Lead
6	Ogbodo Chimdindu Favour	2022514051	Requirements & Research	Stakeholder Interview Officer
7	Ugwu Anthony Chukwuemeka	2022514076	Requirements & Research	Comparative Systems Researcher
8	Nwachukwu Augustine Ifechukwu	2022514043	Requirements & Research	Airline Fare & Policy Researcher
9	Anekwe Favour Uchechukwu	2022514181	Documentation & Writing	Documentation Lead
10	Unachukwu Chimamanda Jessica	2022514050	Documentation & Writing	Technical Writer — Requirements & Design
11	Ekemezie Chibuikem Azuka	2022514099	Documentation & Writing	Technical Writer — Implementation & Testing
12	Elobisi Michael Chukwuebuka	2022514042	Documentation & Writing	Diagrams & Figures Officer
13	Kenechi Richard Michael	2022514179	User Testing & Feedback	User Acceptance Testing Lead
14	Ogheneakoro Ugochukwu Akpevwe	2022514151	User Testing & Feedback	Exploratory Tester — Search & Booking

S/N	Name	Matriculation Number	Sub-team	Principal Role
15	Okeke Isreal Chukwuka	2022514011	User Testing & Feedback	Exploratory Tester — Accounts & Administration
16	Chukwuka Kamsiyochukwu Nelson	2022514185	User Testing & Feedback	Defect Log Recorder
17	Ukohah Onyinye Nkechi	2022514143	Review & Presentation	Usability & Accessibility Reviewer
18	Nwoye David Chiagoziem	2022514172	Review & Presentation	Editorial & Proofreading Reviewer
19	Iheanyi Valentine Kelechi	2022514117	Review & Presentation	Presentation & Demonstration Lead

Declaration. We declare that this project and the accompanying report are the result of our own work, that all sources consulted have been acknowledged in the references, and that the software submitted alongside this report was designed, written and tested by the members of this group. Appendix D records the principal contribution of each member.

Signature	Name	Date
	Group Leader: Ezedimbu Ebube Emmanuel (2022514081)	
	Supervisor: Dr. Alade	

Abstract

Air travel in Nigeria and across West Africa is increasingly booked online, yet the reservation systems that make this possible are large, proprietary and closed. Students of software engineering therefore rarely get the opportunity to study how such a system is put together from the inside. This project addresses that gap by designing and implementing SkyRoute, a working flight booking simulation that reproduces the full commercial reservation journey — searching a live schedule, comparing dynamically priced fares, choosing a seat from an aircraft-specific seat map, entering passenger details, paying, receiving a Passenger Name Record, and later retrieving or cancelling the reservation — together with an administrative console for managing flights, bookings and users.

The system is a client-side web application built with Next.js 16, React 19 and TypeScript, styled with Tailwind CSS. A deliberate constraint was imposed at the outset: the system uses no database and no server-side data store. All state is persisted in the browser's `localStorage`. That constraint shaped the architecture rather than compromising it. A four-layer design was adopted — presentation, application, domain and persistence — in which exactly one module of 102 lines is permitted to touch `localStorage`. Because every layer above it programs against interfaces rather than against the storage mechanism, the persistence layer could be exchanged for a relational database and a REST API without altering a single business rule or interface component.

The delivered system supports one-way, return and multi-city journeys, guest booking without an account, three simulated payment methods, and a user-selectable light and dark appearance. Quality was assured through a layered test strategy centred on 211 automated unit and integration tests written with Vitest, supported by manual system testing, a documented threat model and an accessibility review that honours reduced motion, reduced transparency and increased contrast preferences. All 211 automated tests pass and the codebase type-checks cleanly in TypeScript strict mode.

The report documents the requirements analysis, the architectural and algorithmic design, the implementation, the testing regime and the results, and closes with a candid account of the challenges met — including the defects found and fixed, and the static-analysis findings that remain open — together with a concrete migration path to a server-backed deployment.

Keywords: *flight reservation system, booking simulation, Next.js, React, TypeScript, localStorage, layered architecture, repository pattern, dynamic pricing, seat allocation, accessibility, software testing.*

Table of Contents

Group Members.....	2
Abstract.....	4
CHAPTER ONE: INTRODUCTION.....	11
1.1 Background of the Study.....	11
1.2 Statement of the Problem.....	11
1.3 Aim and Objectives.....	12
1.3.1 Aim.....	12
1.3.2 Specific Objectives.....	12
1.4 Scope of the Project.....	13
1.4.1 Within Scope.....	13
1.4.2 Outside Scope.....	14
1.5 Significance of the Study.....	14
1.6 Limitations of the Study.....	14
1.7 Definition of Terms.....	15
1.8 Organisation of the Report.....	16
CHAPTER TWO: REVIEW OF RELATED SYSTEMS.....	17
2.1 Introduction.....	17
2.2 Evolution of Computerised Reservation.....	17
2.3 Categories of Reservation System.....	17
2.4 Review of Representative Systems.....	18
2.4.1 Amadeus Altéa.....	18
2.4.2 Air Peace direct-sell website.....	18
2.4.3 Wakanow.....	18
2.4.4 Google Flights.....	18
2.5 Comparative Analysis.....	19
2.6 Gaps Identified.....	19
CHAPTER THREE: REQUIREMENTS ANALYSIS.....	21
3.1 Requirements Elicitation.....	21
3.2 Functional Requirements.....	21
3.3 Non-Functional Requirements.....	23
3.4 System Actors.....	24
3.5 Use-Case Model.....	24
3.6 Expanded Use Case: UC-05 Book a Journey.....	26
CHAPTER FOUR: DESIGN APPROACH.....	28
4.1 The Decision That Governs Everything Else.....	28
4.2 Candidate Architectures Considered.....	28
4.3 The Selected Architecture.....	28
4.4 Design Patterns Applied.....	30
CHAPTER FIVE: SYSTEM DESIGN.....	31
5.1 Domain Model.....	31
5.2 Persistence Design.....	32
5.3 Process Design.....	33
5.4 Algorithm Design.....	35

5.4.1 The fare engine.....	35
5.4.2 Seat map derivation.....	36
5.4.3 PNR generation.....	37
5.4.4 Refund calculation.....	37
5.5 Interface and Appearance Design.....	38
5.5.1 Design tokens.....	38
5.5.2 Colour and contrast.....	38
5.5.3 Motion and preference.....	39
5.6 Security Design.....	39
CHAPTER SIX: DEVELOPMENT METHODOLOGY.....	42
6.1 Selecting a Methodology.....	42
6.2 Justification for an Iterative Approach.....	42
6.3 Iteration Plan.....	43
6.4 Team Organisation.....	44
6.5 Definition of Done.....	45
6.6 Collaboration Practices.....	45
CHAPTER SEVEN: TECHNOLOGIES AND TOOLS.....	47
7.1 Programming Language.....	47
7.2 Framework.....	47
7.3 Persistence Mechanism.....	48
7.4 Complete Technology Stack.....	49
7.5 A Note on Dependency Discipline.....	50
CHAPTER EIGHT: IMPLEMENTATION.....	51
8.1 Overview.....	51
8.2 The Persistence Layer.....	52
8.3 The Domain Layer.....	52
8.3.1 The fare engine.....	52
8.3.2 Seat map derivation.....	53
8.4 The Application Layer.....	53
8.4.1 Authorisation, and the guest case.....	54
8.5 The Presentation Layer.....	55
8.5.1 Server rendering and the hydration problem.....	55
8.5.2 Explanatory motion.....	55
8.5.3 The appearance system.....	56
8.6 Development Practices Observed.....	56
CHAPTER NINE: TESTING.....	57
9.1 Testing Strategy.....	57
9.1.1 Test design techniques.....	57
9.2 Automated Test Suite.....	57
9.3 Illustrative Test Cases.....	58
9.4 Requirements Traceability Matrix.....	59
9.5 System Test Cases.....	60
9.6 Security and Accessibility Testing.....	62
9.7 Static Analysis.....	63
CHAPTER TEN: RESULTS AND DISCUSSION.....	65
10.1 What Was Delivered.....	65

10.2 Evaluation Against the Objectives.....	66
10.3 Discussion.....	66
10.3.1 The constraint was productive.....	66
10.3.2 Deriving state eliminates a class of defect.....	67
10.3.3 Testing the failure paths mattered most.....	67
10.3.4 What a database was actually providing.....	67
10.3.5 Accessibility is cheaper when it is not retrofitted.....	68
10.3.6 Where the codebase is uneven.....	68
10.4 Comparison with the Reviewed Systems.....	68
10.5 Limitations Observed in Practice.....	68
CHAPTER ELEVEN: CHALLENGES ENCOUNTERED.....	69
11.1 Technical Challenges.....	69
11.1.1 Server rendering against client-only storage.....	69
11.1.2 Concurrent seat selection.....	69
11.1.3 Growing a booking from one flight to several.....	69
11.1.4 Fare arithmetic across legs.....	70
11.1.5 Naming the failure in a multi-leg booking.....	70
11.1.6 An appearance the operating system fought.....	70
11.2 Process and Team Challenges.....	70
11.2.1 A single developer as a bottleneck.....	70
11.2.2 Uneven attention across the codebase.....	71
11.2.3 Estimating unfamiliar work.....	71
11.3 Defect Log.....	71
CHAPTER TWELVE: CONCLUSION.....	73
12.1 Summary.....	73
12.2 Contributions.....	73
12.3 Lessons Learned.....	74
12.4 Concluding Remarks.....	74
CHAPTER THIRTEEN: RECOMMENDATIONS.....	75
13.1 Recommendations for Practice.....	75
13.2 Migration to a Server-Backed Deployment.....	75
13.3 Immediate Technical Recommendations.....	76
13.4 Functional Enhancements.....	76
13.5 Recommendations for Academic Use.....	77
REFERENCES.....	78
APPENDIX A: INSTALLATION AND USER GUIDE.....	80
A.1 Prerequisites.....	80
A.2 Installation.....	80
A.3 Available Commands.....	80
A.4 Demonstration Accounts.....	80
A.5 Walkthrough: Making a Booking.....	81
A.6 Walkthrough: Managing a Booking.....	81
A.7 Troubleshooting.....	82
APPENDIX B: PROJECT STRUCTURE.....	83
APPENDIX C: VERIFICATION EVIDENCE.....	84

C.1 Automated Test Suite.....	84
C.2 Type Check.....	84
C.3 Static Analysis.....	84
APPENDIX D: CONTRIBUTION RECORD.....	85

List of Tables

Table 1.1	Definition of technical terms.....	15
Table 2.1	Categories of system in the air travel distribution chain.....	18
Table 2.2	Comparison of reviewed systems against the proposed system.....	19
Table 3.1	Functional requirements.....	23
Table 3.2	Non-functional requirements.....	24
Table 3.3	System actors.....	24
Table 3.4	Use-case inventory mapped to functional requirements.....	26
Table 3.5	Expanded use case UC-05: Book a journey.....	27
Table 4.1	Comparison of candidate architectural styles.....	28
Table 4.2	Responsibilities and permitted dependencies of each layer.....	30
Table 5.1	Entity attribute definitions.....	32
Table 5.2	localStorage key schema.....	33
Table 5.3	Advance-purchase pricing factor.....	35
Table 5.4	Demand factor derived from cabin load.....	36
Table 5.5	Cancellation refund schedule.....	37
Table 5.6	Colour tokens and their contrast ratios.....	39
Table 5.7	Threat model and implemented mitigations.....	40
Table 6.1	Comparison of software development methodologies.....	42
Table 6.2	Iteration plan and deliverables.....	43
Table 6.3	Sub-team composition and responsibilities.....	45
Table 7.1	Programming language alternatives considered.....	47
Table 7.2	Framework alternatives considered.....	48
Table 7.3	Persistence alternatives considered.....	48
Table 7.4	Complete technology stack.....	49
Table 8.1	Source file inventory.....	51
Table 9.1	Automated test suite summary.....	58
Table 9.2	Requirements traceability matrix.....	60
Table 9.3	System test cases and results.....	62
Table 9.4	Security test results.....	63
Table 9.5	Static analysis findings outstanding.....	64
Table 10.1	Delivered system by the numbers.....	65
Table 10.2	Objectives evaluated against outcomes.....	66
Table 11.1	Defect log.....	72
Table 13.1	Migration plan to a server-backed deployment.....	76

List of Figures

Figure 3.1	Use-case diagram.....	25
Figure 4.1	Four-layer system architecture.....	29
Figure 5.1	Entity-relationship diagram.....	31

Figure 5.2 Booking process flowchart.....	10
Figure 5.3 Sequence diagram for seat reservation.....	10
Figure 5.4 Site map and navigation structure.....	10
Figure 8.1 Module dependency graph.....	10
Figure 9.1 The test strategy as applied.....	10
Figure 9.2 Output of the automated suite.....	10

CHAPTER ONE: INTRODUCTION

1.1 Background of the Study

The commercial aviation industry runs on software. Before a passenger ever reaches an aircraft, a chain of computer systems has quoted a fare, held an inventory of seats, captured identity documents, authorised a payment, issued a record locator and, in many cases, allocated a specific seat. These systems — known collectively as Computer Reservation Systems, and in their federated form as Global Distribution Systems — are among the oldest continuously operating transaction-processing systems in existence. American Airlines and IBM's SABRE, brought into full service in 1964, is generally credited as the origin of the field, and its descendants still process a substantial share of the world's air bookings.

In Nigeria the adoption of online booking has accelerated sharply over the past decade. Domestic carriers such as Air Peace, Ibom Air, United Nigeria and Green Africa all operate direct-sell websites, and online travel agencies aggregate them alongside international carriers. For a passenger travelling from Lagos to Abuja, the entire transaction — comparing several carriers, choosing a departure time, selecting a window seat and paying by card or bank transfer — is now routinely completed on a mobile phone in a few minutes.

What is straightforward for the passenger is not straightforward for the engineer. A reservation system must reconcile several difficult concerns at once: a perishable inventory, in which an unsold seat becomes worthless the moment the aircraft departs; a pricing model that varies continuously with time and demand; a concurrency problem, in which many users compete for the same finite seats; a security obligation, since the system handles identity documents and payment instruments; and an accessibility obligation, since a booking site is for everybody.

Students of software engineering are rarely able to study these concerns from the inside. Production reservation systems are proprietary, their source is closed, and their published documentation is written for integrators rather than for learners. There is therefore genuine educational value in constructing a simulation: a system that reproduces the structure and the difficult parts of a reservation system faithfully enough to be instructive, while remaining small enough for a student group to design, build, test and reason about in a single semester.

1.2 Statement of the Problem

The problem this project addresses has two faces, one pedagogical and one technical.

The pedagogical problem is one of access. A student who wishes to understand how seat inventory is prevented from being oversold, how a fare responds to how full a cabin is, or how a Passenger Name Record is generated and made collision-resistant, has no realistic system to examine. Textbook treatments describe these mechanisms in the abstract; commercial systems implement them behind a closed interface. What is missing is a complete, readable, working system whose every design decision can be inspected and defended.

The technical problem is one of constraint. Most educational booking projects reach immediately for a database and a server, and in doing so they distribute the interesting logic across a network boundary where it becomes harder to see. A more demanding — and more revealing — question is whether a substantial transactional application can be built with no database at all, storing its entire state in the browser. If it can, then the discipline that makes it possible is precisely the discipline that good architecture is supposed to supply: the separation of business rules from the mechanism that stores them.

This project therefore poses and answers a specific question: can a realistic flight booking system, complete with multi-leg journeys, seat inventory, dynamic pricing, concurrency control, authentication, authorisation and administration, be implemented entirely on the client using browser `localStorage` — and what must the architecture look like for that to be feasible?

1.3 Aim and Objectives

1.3.1 Aim

The aim of this project is to design, develop, test and document a functional flight booking simulation system that demonstrates the practical application of software engineering principles — architectural layering, domain modelling, algorithm design, secure programming, accessible interface design and systematic testing — within the deliberate constraint of client-side-only persistence.

1.3.2 Specific Objectives

To achieve this aim, the project set out to:

1. Elicit and specify a complete set of functional and non-functional requirements for a flight reservation system, expressed in a form that can be traced through design, implementation and testing.
2. Design a layered software architecture in which business rules are wholly independent of the storage mechanism, so that the persistence layer can be replaced without disturbing the rest of the system.
3. Model the problem domain — airports, flights, aircraft cabins, seats, passengers, journey segments, bookings, payments and user accounts — and define the relationships and constraints between these entities.
4. Design and implement a dynamic fare engine in which price responds to how far in advance a booking is made, how full the requested cabin is, the passenger's age band and the cabin class selected.
5. Implement an interactive, aircraft-specific seat map that derives availability from the booking record rather than storing it separately, thereby making oversold seats structurally impossible.
6. Implement the complete booking transaction for one-way, return and multi-city journeys, with correct behaviour on the failure path as well as the success path, and issuance of a unique Passenger Name Record.

7. Support booking both with an account and as a guest, and allow either to be retrieved and cancelled afterwards under the appropriate authorisation rule.
8. Implement an administrative module providing schedule management, booking oversight, a user directory and aggregate operational reporting.
9. Design a coherent visual identity and interface that meets recognised accessibility guidance, including a user-selectable appearance and respect for the reader's motion, transparency and contrast preferences.
10. Verify the system through a layered test strategy supported by an automated regression suite and a requirements traceability matrix.
11. Document the entire process in a technical report that makes the design decisions, the trade-offs and the limitations explicit.

1.4 Scope of the Project

1.4.1 Within Scope

- A searchable schedule spanning 21 days, 16 airports, 37 route templates and six aircraft types, generated on first run and thereafter editable by an administrator.
- One-way, return and multi-city journey construction, with each leg priced and seated independently and settled as a single booking.
- Search by origin, destination, date, cabin class and party size, with sorting, filtering and suggested alternative dates.
- Account registration and authentication, with passwords stored only as salted, iteratively hashed digests, alongside guest booking for users who prefer not to register.
- An interactive seat map generated from the cabin configuration of the specific aircraft type operating each flight, distinguishing window, aisle, middle and exit-row seats.
- A dynamic fare engine with an itemised quotation including value-added tax and a once-per-booking service charge.
- Passenger data capture with an age-band cross-check, and passport capture conditional on the route being international.
- Three simulated payment methods — card, bank transfer and wallet — with genuine Luhn validation, brand detection and expiry checking on the card path.
- Booking confirmation with a six-character Passenger Name Record and a printable electronic ticket.
- Retrieval and cancellation by record locator and surname, with an automatic refund computed on a published sliding scale.
- An administrative console with five modules: operational overview, flight management, booking management, user directory and system maintenance.

- A light and dark appearance the reader can select, and an interface that honours reduced-motion, reduced-transparency and increased-contrast preferences.
- A regression suite of 211 automated tests.

1.4.2 Outside Scope

- Real payment processing. No payment gateway is integrated and no money moves. Card details are validated and then discarded; only a masked representation of the number is retained.
- Electronic mail. Confirmation messages are described in the interface but no message is transmitted, as this requires a mail server.
- Connecting itineraries with through-fares. Multi-city journeys are supported, but each leg is priced independently rather than as a constructed through-fare.
- Live airline inventory. The schedule is fictional and self-generated; no GDS, NDC or airline API is contacted.
- Online check-in, boarding-pass issuance, ancillary sales and loyalty accrual.
- Cross-device synchronisation. Data resides in one browser on one machine — an inherent consequence of the localStorage constraint, discussed in Section 10.5.

1.5 Significance of the Study

The project makes three contributions. Academically, it provides a complete, readable reference implementation of a transactional reservation system in which every layer, every algorithm and every trade-off is documented and can be inspected — something not otherwise available to students. Technically, it demonstrates that architectural discipline, and specifically the isolation of the persistence mechanism behind a single narrow interface, is what allows an unusual infrastructural constraint to be absorbed without contaminating the rest of the system. Practically, it produces a working artefact that runs with no database server and no network connection once loaded, and can therefore be demonstrated and assessed anywhere.

1.6 Limitations of the Study

- **Persistence is per-browser.** Data written in one browser is invisible in another, and clearing browsing data erases the entire system state. There is no backup.
- **Storage capacity is bounded.** Browsers typically allocate around 5 MB per origin. The seeded schedule occupies a small fraction of that, but the ceiling is real.
- **Client-side security is inherently limited.** Password hashing protects against casual inspection and credential reuse, but a user with developer tools can read and modify stored data, including their own role. Genuine authentication requires a server; Section 13.2 sets out the design that would provide it.

- **Concurrency control is best-effort.** Seat conflicts between browser tabs are detected and refused at the moment of writing, but `localStorage` offers no transactional guarantee comparable to a database's serialisable isolation level.
- **The schedule is fictional.** Airlines, flight numbers, timings and fares are plausible but invented. No booking made in this system is valid for travel.

1.7 Definition of Terms

Term	Definition
PNR (Passenger Name Record)	The record of a reservation held by an airline, identified by a six-character alphanumeric locator quoted by the passenger at check-in.
Segment	One flight within a journey. A one-way booking has one segment; a return has two; a multi-city booking has as many as were built.
Cabin class	The service class in which a passenger travels — Economy, Business or First — each occupying a distinct physical section of the aircraft.
Load factor	The proportion of fitted seats that have been sold, between 0 and 1. A key input to the dynamic fare engine.
Advance purchase	The number of whole days between booking and departure. Fares generally rise as this interval shrinks.
Seat map	A plan view of the aircraft cabin showing each seat's position, characteristics and availability.
Exit row	A seat row adjacent to an emergency exit, offering extra legroom and attracting a higher selection fee.
Luhn algorithm	A checksum formula used to validate the structural correctness of a payment card number before it is submitted to a network.
localStorage	A browser API providing synchronous, origin-scoped, string-keyed persistent storage that survives reloads and browser restarts.
Repository pattern	A design pattern that interposes a collection-like abstraction between business logic and the mechanism that stores data.
Hydration	In React, the process by which client-side JavaScript attaches behaviour to server-rendered markup. A mismatch between the two produces a hydration error.
Salt	Random data combined with a password before hashing, so identical passwords produce different digests and precomputed tables are useless.
Design token	A named value — a colour, a duration, a radius — referenced by name rather than by literal, so one edit changes every use.

Table 1.1 Definition of technical terms used in this report.

1.8 Organisation of the Report

Chapter Two reviews existing reservation systems and identifies the gaps this project addresses. Chapter Three presents the requirements analysis. Chapter Four sets out the design approach and the architecture. Chapter Five documents the system design in detail: the domain model, the persistence schema, the process and algorithm designs, the interface and appearance system, and the security design. Chapter Six explains the development methodology and the organisation of the nineteen-member group. Chapter Seven justifies the choice of language, framework, libraries and tools. Chapter Eight describes the implementation with annotated extracts. Chapter Nine sets out the testing strategy and results. Chapter Ten presents and discusses the results. Chapter Eleven records the challenges encountered. Chapter Twelve concludes, and Chapter Thirteen offers recommendations.

CHAPTER TWO: REVIEW OF RELATED SYSTEMS

2.1 Introduction

This chapter surveys the landscape into which SkyRoute fits. It begins with the historical development of computerised reservation, examines the categories of system in current use, reviews four representative systems, and closes by identifying the gaps that motivated the present work.

2.2 Evolution of Computerised Reservation

Before automation, airline seat inventory was maintained on physical card indexes in a central reservations office. An agent telephoned the office, an operator located the card for the flight and date, confirmed a seat was free, and marked it. The process was slow, error-prone and impossible to scale.

SABRE demonstrated the central insight that still governs the field: seat inventory is a shared, contended, perishable resource, and correctness depends on serialising access to it. Competing carriers built equivalents, and by the 1980s these systems had been opened to travel agents, creating the Global Distribution System as a category — a neutral intermediary aggregating inventory from many carriers.

The commercial internet changed the distribution model rather than the underlying inventory logic. Airlines built direct-sell websites, online travel agencies emerged as digital intermediaries, and low-cost carriers pioneered the unbundled fare in which the seat, the bag and the meal are priced separately. More recently, IATA's New Distribution Capability has begun to replace the venerable EDIFACT teletype protocols with XML-based messaging. Throughout all of this, the seat-inventory problem has remained recognisably the problem SABRE solved.

2.3 Categories of Reservation System

Category	Description	Representative examples
Global Distribution System	Neutral intermediaries aggregating inventory from many carriers and distributing it to agencies and online travel agents.	Amadeus, Sabre, Travelport
Airline direct-sell platform	A carrier's own website and mobile application, selling only that carrier's inventory and its codeshare partners'.	airpeace.com, ibomair.com, britishairways.com
Online travel agency	Consumer-facing retailers that search across many carriers, add a service fee and issue tickets on the carriers' behalf.	Wakanow, Travelstart, Expedia
Metasearch engine	Systems that search and compare but do not sell, redirecting the user to a carrier or agency to complete the transaction.	Google Flights, Skyscanner, Kayak

Category	Description	Representative examples
Departure control system	Post-booking systems handling check-in, airport seat assignment, baggage acceptance and boarding.	Amadeus Altéa DCS, SITA Horizon

Table 2.1 Categories of system in the air travel distribution chain.

SkyRoute simulates an airline direct-sell platform with an embedded administrative back office. This category was chosen because it contains the complete transaction from search to cancellation within a single system boundary, making it the most instructive to model in its entirety.

2.4 Review of Representative Systems

2.4.1 Amadeus Altéa

Amadeus Altéa is a full passenger service system used by more than 200 carriers. Its architecture separates three concerns that this project also separates: inventory (what seats exist and which are sold), reservation (who has booked what) and departure control (who actually boards). Its relevance here is architectural: the separation of inventory from reservation is precisely the separation that lets SkyRoute derive seat availability from the booking record rather than maintaining a parallel availability table that could drift out of step.

2.4.2 Air Peace direct-sell website

Air Peace operates the largest domestic network in Nigeria and sells through a conventional direct-sell website. The booking flow is a multi-step wizard and integrates Nigerian payment methods including card and bank transfer. It is a useful reference for the expectations of the Nigerian market: fares quoted in naira, phone numbers in the local format, and a payment step that offers transfer alongside card — all three of which SkyRoute reproduces. A limitation observed on several Nigerian carrier sites is that interactive seat selection at the time of booking is either absent or deferred to check-in; SkyRoute deliberately includes it, because seat inventory is one of the more interesting problems in the domain.

2.4.3 Wakanow

Wakanow is a Nigerian online travel agency aggregating multiple carriers alongside hotels and holiday packages. Its value to this review is its result-comparison interface: results are presented as a scannable list with sorting and filtering, and the total for the whole party is shown alongside the per-passenger fare. SkyRoute adopts both conventions. As an intermediary, however, Wakanow does not expose seat maps or the fare construction, and the administrative side of the business is entirely hidden — reinforcing the case for a simulation that includes the back office.

2.4.4 Google Flights

Google Flights is a metasearch engine notable for the speed and clarity of its comparison interface and for surfacing price history and calendar-based comparison. It does not sell tickets, hold inventory or

process payments. Its contribution to this project's design is the alternative-date suggestion: when a search returns nothing, offering nearby dates that do have availability is materially more helpful than an empty result. SkyRoute implements this behaviour.

2.5 Comparative Analysis

Capability	Amadeus	Air Peace	Wakanow	Google Flights	SkyRoute
Multi-carrier search	Yes	No	Yes	Yes	Yes
Interactive seat map at booking	Yes	Limited	No	No	Yes
Multi-city journey construction	Yes	Limited	Yes	Yes	Yes
Dynamic fare construction	Yes	Yes	Yes	Display only	Yes
Itemised fare breakdown	Yes	Partial	Partial	No	Yes
Guest booking without an account	N/A	Yes	Yes	N/A	Yes
Self-service cancellation with refund quote	Yes	Partial	Partial	No	Yes
Administrative back office	Yes	Internal only	Internal only	N/A	Yes, exposed
Source available for study	No	No	No	No	Yes
Runs without a server or database	No	No	No	No	Yes

Table 2.2 Comparison of reviewed systems against the proposed system.

2.6 Gaps Identified

1. **Opacity.** Every system reviewed is closed. None can be read, modified or reasoned about by a student, and none documents why its designers chose one approach over another.
2. **Hidden administration.** The operational side — schedule management, booking oversight, revenue reporting — is invisible from outside, yet it is half of what a reservation system does.
3. **Infrastructure dependence.** All require servers, databases and network connectivity, which makes them unusable as teaching artefacts in constrained environments.
4. **Undocumented pricing.** Fares appear without explanation. The relationship between booking date, cabin occupancy and price is commercially sensitive and never disclosed, so students cannot study it.

SkyRoute addresses all four: its source is open and annotated, its administrative console is a first-class part of the delivered system, it runs with no server or database whatsoever, and its fare engine is a documented, unit-tested set of pure functions whose every factor is stated in this report.

CHAPTER THREE: REQUIREMENTS ANALYSIS

3.1 Requirements Elicitation

Requirements were gathered using four complementary techniques, chosen because no single technique gives a complete picture of a domain the team did not begin as experts in.

- **Analysis of existing systems.** The team walked the complete booking journey on four production websites, recording each screen, field, validation message and decision point. This produced the initial functional inventory.
- **Structured interviews.** Frequent travellers were interviewed about what they value and what frustrates them when booking online. Three themes recurred: the total price for the whole party should be visible early rather than at the final step; seat choice matters to most travellers; and cancellation terms should be stated before the cancellation is committed, not after.
- **Document study.** Published carrier conditions of carriage were examined for the structure of fare rules, refund schedules and passenger age bands, which informed the pricing and refund algorithms.
- **Prototyping.** Low-fidelity wireframes of the search, seat map and payment screens were reviewed with prospective users, which surfaced requirements the interviews had not — notably alternative-date suggestions, automatic seat assignment, and the demand for booking without creating an account.

3.2 Functional Requirements

Each requirement carries an identifier used throughout the report and in the traceability matrix in Section 9.4. Priority follows the MoSCoW convention.

ID	Requirement	Priority
FR-01	The system shall allow a visitor to register an account with a full name, email address, Nigerian phone number and password, and shall reject an email address already registered.	Must
FR-02	The system shall authenticate a registered user by email and password, returning an identical error message whether the email is unknown or the password is wrong.	Must
FR-03	The system shall allow a user to search for flights by origin, destination, departure date, cabin class and party size.	Must
FR-04	The system shall compute a fare for each result that varies with the advance-purchase interval, the load factor of the requested cabin, the passenger's age band and the cabin class.	Must
FR-05	The system shall present an interactive seat map derived from the cabin configuration of the aircraft operating the flight, distinguishing window, aisle, middle and exit-row seats.	Must

ID	Requirement	Priority
FR-06	The system shall prevent any seat from being allocated to more than one passenger, re-verifying availability at the moment the booking is written.	Must
FR-07	The system shall capture a title, first name, surname, date of birth, gender and — for international routes only — a passport number for each passenger, and shall verify the date of birth is consistent with the fare type.	Must
FR-08	The system shall simulate payment authorisation by card, bank transfer or wallet, validating card numbers with the Luhn algorithm, detecting the brand and rejecting an expired card.	Must
FR-09	The system shall issue a unique six-character Passenger Name Record on successful payment and shall write no booking record when payment fails.	Must
FR-10	The system shall present a printable itinerary showing every flight on the journey, all passengers, their seat allocations, the itemised fare and the payment reference.	Must
FR-11	The system shall list all bookings belonging to the signed-in user, filterable by upcoming, past and cancelled.	Must
FR-12	The system shall allow a booking to be retrieved by Passenger Name Record together with a passenger surname, without requiring the user to sign in.	Must
FR-13	The system shall allow a booking to be cancelled by its owner, by an administrator, or by a guest who can produce the surname on it, shall quote the refund before committing, and shall compute it on a published sliding scale.	Must
FR-14	The system shall return cancelled seats to the available pool on every segment of the journey.	Must
FR-15	The system shall provide an administrative console accessible only to users holding the administrator role.	Must
FR-16	The administrative console shall allow flights to be created, have their status changed and be deleted, and shall refuse deletion of any flight carrying confirmed bookings.	Must
FR-17	The administrative console shall list all bookings, searchable by record locator, passenger name or contact email, and shall allow an administrator to cancel any booking.	Should
FR-18	The administrative console shall report aggregate statistics: flights scheduled, bookings confirmed and cancelled, passengers carried, registered users, gross and net revenue, and the busiest routes.	Should
FR-19	The system shall support one-way, return and multi-city journeys, pricing and seating each leg independently while settling them as a single booking.	Must
FR-20	The system shall allow a booking to be made as a guest, without an account, and shall keep such a booking out of every account's booking list.	Should
FR-21	The system shall offer alternative departure dates when a search returns no results but the route is served on other dates.	Should

ID	Requirement	Priority
FR-22	The system shall allow results to be sorted and filtered, and shall offer automatic seat assignment as an alternative to manual selection.	Should
FR-23	The system shall seed a schedule automatically on first run and allow an administrator to reset the system to that seeded state.	Should
FR-24	The system shall offer a light and a dark appearance, defaulting to the operating system preference and remembering an explicit choice.	Should

Table 3.1 Functional requirements.

3.3 Non-Functional Requirements

ID	Category	Requirement	Acceptance criterion
NFR-01	Performance	Search shall return results without perceptible delay on a schedule of the seeded size.	Median search completes well inside 200 ms on a mid-range laptop.
NFR-02	Performance	The seat map shall render without perceptible delay for the largest aircraft modelled.	A wide-body map renders inside 300 ms.
NFR-03	Security	Passwords shall never be stored, transmitted or displayed in plain text.	No plain-text password appears in stored state; verified by automated test.
NFR-04	Security	Password verification shall not leak information through its execution time.	Comparison is constant-time over equal-length digests; verified by test.
NFR-05	Security	Full payment card numbers shall never be persisted.	Only the last four digits are retained; verified by test.
NFR-06	Security	Administrative operations shall be refused for non-administrators regardless of interface state.	Authorisation enforced in the repository layer; verified by test.
NFR-07	Reliability	Corrupted or hand-edited stored state shall not crash the application.	Malformed JSON degrades to a safe empty state; verified by test.
NFR-08	Reliability	A failed payment shall leave no partial booking behind.	Storage is unchanged after a declined payment; verified by test.
NFR-09	Reliability	A multi-leg journey shall be refused in its entirety if any single leg is unbookable.	No partial journey is written; verified by test.
NFR-10	Usability	The booking journey shall be completable by a first-time user without instruction.	Acceptance participants complete a booking unaided.
NFR-11	Usability	Validation errors shall identify the field at fault and state how to correct it.	Every field-level error names the field and the expected format.

ID	Category	Requirement	Acceptance criterion
NFR-12	Accessibility	All interactive controls shall be reachable and operable by keyboard alone, with a visible focus indicator.	Full booking journey completed using only the keyboard.
NFR-13	Accessibility	The interface shall honour the reader's reduced-motion, reduced-transparency and increased-contrast preferences.	Each preference has a dedicated media query in the stylesheet.
NFR-14	Accessibility	Body text shall meet WCAG 2.1 AA contrast in both appearances.	Every text token measured at 4.5:1 or better against its surface.
NFR-15	Responsiveness	The interface shall be usable on viewports from 320 px to 1920 px wide.	No horizontal scrolling or clipped content at any tested width.
NFR-16	Maintainability	Business logic shall be independent of the storage mechanism.	Exactly one module references the localStorage API.
NFR-17	Maintainability	The codebase shall be statically type-checked with no errors.	TypeScript strict mode compiles clean.
NFR-18	Portability	The system shall run with no database server and no network connection after loading.	Application functions fully offline once loaded.

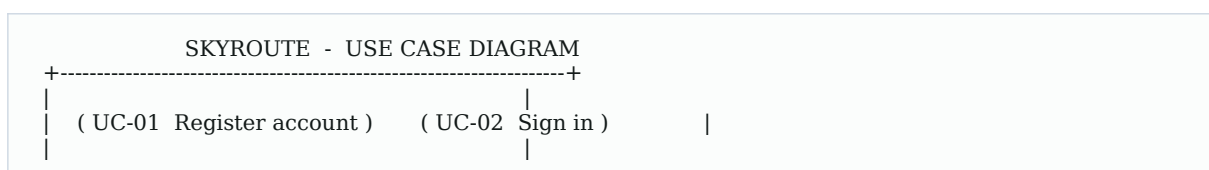
Table 3.2 Non-functional requirements.

3.4 System Actors

Actor	Description	Principal goals
Visitor	An unauthenticated user of the system.	Search for flights, compare fares, book as a guest, retrieve an existing booking, register an account.
Customer	An authenticated user holding the customer role.	Complete a booking, view and print itineraries, cancel a booking and receive a refund quotation.
Administrator	An authenticated user holding the administrator role.	Manage the schedule, oversee all bookings, review the user directory, monitor statistics, reset the system.
Payment simulator	An internal component standing in for an external payment gateway.	Authorise or decline a simulated transaction and return a transaction reference.

Table 3.3 System actors.

3.5 Use-Case Model



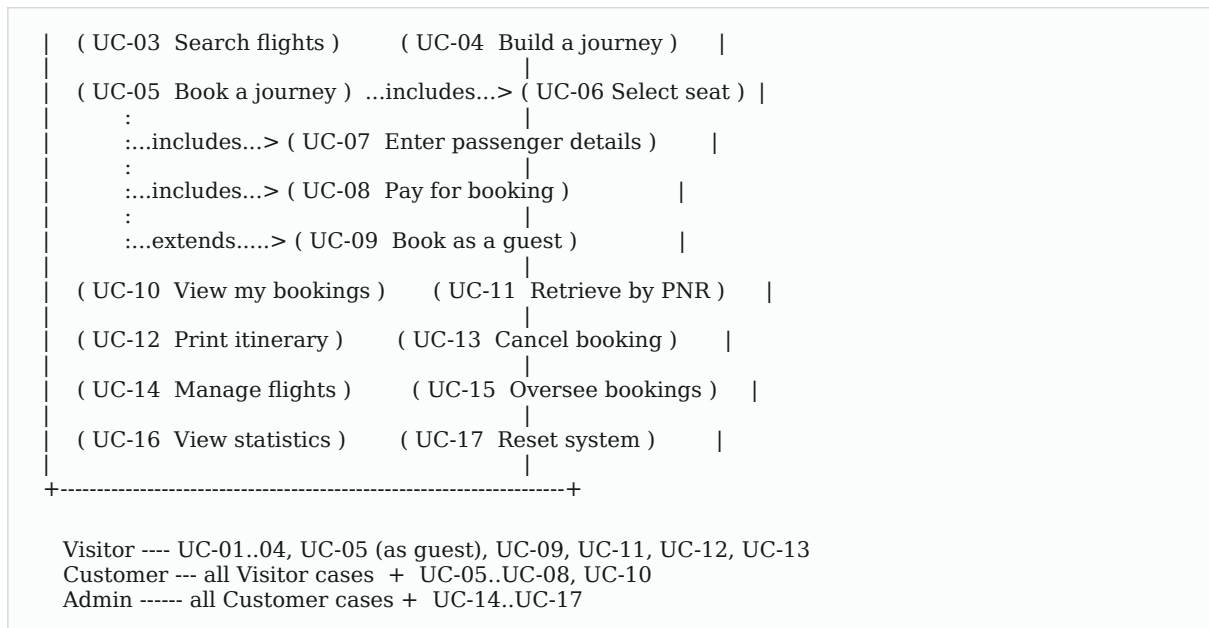


Figure 3.1 Use-case diagram showing actors and their permitted use cases.

ID	Use case	Primary actor	Requirements
UC-01	Register an account	Visitor	FR-01
UC-02	Sign in	Visitor	FR-02
UC-03	Search for flights	Visitor	FR-03, FR-04, FR-21
UC-04	Build a one-way, return or multi-city journey	Visitor	FR-19
UC-05	Book a journey	Customer	FR-05 – FR-10
UC-06	Select a seat	Customer	FR-05, FR-06, FR-22
UC-07	Enter passenger details	Customer	FR-07
UC-08	Pay for a booking	Customer	FR-08, FR-09
UC-09	Book as a guest	Visitor	FR-20
UC-10	View my bookings	Customer	FR-11
UC-11	Retrieve a booking by PNR	Visitor	FR-12
UC-12	Print an itinerary	Visitor	FR-10
UC-13	Cancel a booking	Customer / Visitor	FR-13, FR-14
UC-14	Manage the flight schedule	Administrator	FR-15, FR-16
UC-15	Oversee all bookings	Administrator	FR-17
UC-16	View operational statistics	Administrator	FR-18

ID	Use case	Primary actor	Requirements
UC-17	Reset the system	Administrator	FR-23

Table 3.4 Use-case inventory mapped to functional requirements.

3.6 Expanded Use Case: UC-05 Book a Journey

The central use case is specified in full, since it is the transaction on which the correctness of the whole system depends.

Field	Specification
Use case ID	UC-05
Name	Book a journey
Primary actor	Customer, or Visitor booking as a guest
Stakeholders	Traveller (wants a confirmed seat on every leg at a fair price); airline operator (wants no seat sold twice and every booking paid for)
Preconditions	One or more flights have been selected. Every selected flight is scheduled, has not departed, and has sufficient available seats in the requested cabin.
Postconditions (success)	A booking exists with status confirmed and a unique PNR, carrying one segment per flight. Selected seats are marked occupied on every segment. A successful payment record is attached.
Postconditions (failure)	No booking record exists. No seat has changed state on any leg. Stored state is byte-for-byte unchanged.
Trigger	The traveller confirms their journey selection and proceeds to book.
Main success scenario	1. The system displays the seat map for the requested cabin on the first leg. 2. The traveller selects one seat per fare-paying passenger, or requests automatic assignment. 3. Steps 1 and 2 repeat for each remaining leg of the journey. 4. The traveller enters the details of each passenger and the booking contact information. 5. The system validates every field and cross-checks each date of birth against the fare type. 6. The system displays the combined itemised fare for the whole journey. 7. The traveller chooses a payment method and supplies the details it requires. 8. The system validates those details for the chosen method. 9. The system re-verifies seat availability on every leg against current stored state. 10. The system authorises the simulated payment. 11. The system generates a PNR that does not collide with any existing locator. 12. The system writes the booking and displays the printable itinerary.
Extension 9a — seat taken meanwhile	9a1. Another booking has claimed one of the selected seats since the map was

Field	Specification
	rendered. 9a2. The system refuses the booking and names the flight and the conflicting seats. 9a3. The system refreshes the seat map and returns the traveller to seat selection. 9a4. No booking is written and no payment is taken.
Extension 10a — payment declined	10a1. The simulated gateway declines the transaction. 10a2. The system reports the decline and states explicitly that no seats have been reserved. 10a3. Stored state is unchanged; the traveller may retry or abandon.
Extension 5a — invalid passenger data	5a1. One or more fields fail validation. 5a2. The system marks each offending field and states the expected format. 5a3. The traveller corrects the data; the scenario resumes at step 5.
Special requirements	Steps 9 to 12 must be atomic with respect to stored state: either all take effect or none does, across every leg of the journey.
Frequency	The most frequently executed use case in the system.

Table 3.5 Expanded use case UC-05: Book a journey.

CHAPTER FOUR: DESIGN APPROACH

4.1 The Decision That Governs Everything Else

The design approach is the most consequential decision in this project, because the chosen persistence mechanism — browser `localStorage` — is unusual, limited and likely to be replaced. An architecture that allowed that mechanism to leak into the business rules would produce a system that could never be migrated. The approach was therefore selected specifically for its ability to isolate that decision.

4.2 Candidate Architectures Considered

Style	Description	Assessment for this project
Monolithic / unlayered	All logic written directly into page components; storage accessed wherever convenient.	Rejected. Fastest to start, but storage calls would appear in dozens of components, making the <code>localStorage</code> decision irreversible and the business rules untestable without a DOM.
Layered (n-tier)	Horizontal layers with strictly downward dependencies: presentation, application, domain, persistence.	Selected. Confines storage to one module, keeps the rules pure and testable, and is a well-understood style that a reader can follow without explanation.
Microservices	Independently deployable services communicating over a network.	Rejected. There is no server, so there is nothing to distribute. The operational complexity would be entirely unjustified at this scale.
Event-driven / CQRS	Commands and queries separated; state derived by replaying an event log.	Rejected as the overall style, though its central idea was adopted locally: seat availability is derived from the booking record rather than stored, which is a read-model projection in miniature.
Client-server with REST API	Browser client calling a server that owns the database.	Rejected by the project constraint, but retained as the explicit migration target. Section 13.2 sets out the path.

Table 4.1 Comparison of candidate architectural styles.

4.3 The Selected Architecture

A four-layer client-side architecture was adopted, combined with the Model-View-Controller separation that React and the Next.js App Router naturally encourage. Dependencies point strictly downwards: a layer may call the layer beneath it and may not call the layer above it. The domain layer, at the centre, depends on nothing at all.

```
+=====
=====+
```

The load-bearing design decision

Exactly one module — `lib/storage.ts`, 102 lines — references the `localStorage` API. Every layer above it works with plain JavaScript objects.

The consequence: replacing `localStorage` with a REST API and a relational database requires rewriting that one file and making the repository functions asynchronous. Not one domain rule, not one component and not one test of business logic would need to change.

This is the difference between a constraint that shapes an architecture and a constraint that corrupts one.

Page 29 of 86

			authorisation rules
Application	lib/repository.ts	Domain, Persistence	Import React or reference the DOM
Domain	pricing, seats, validation, auth, ids, types, format	Nothing	Perform any I/O, touch storage, or import from any other layer
Persistence	lib/storage.ts	Nothing	Contain any business rule or interpret the data it stores

Table 4.2 Responsibilities and permitted dependencies of each layer.

4.4 Design Patterns Applied

- **Repository.** lib/repository.ts presents a collection-like interface over stored state. Callers ask for bookings; they never ask for a storage key.
- **Adapter.** lib/storage.ts adapts the string-only localStorage API to typed object reads and writes, absorbing serialisation and error handling.
- **Result object.** Every mutating operation returns a discriminated union — { ok: true, data } or { ok: false, error } — rather than throwing. Callers branch on a value instead of wrapping every call in try/catch, and TypeScript's narrowing makes it impossible to read data without first checking ok.
- **Derived read model.** Seat availability is computed from bookings on demand and never stored. Two representations that could disagree are reduced to one that cannot.
- **Adapter for legacy shape.** bookingSegments() widens a single-flight booking into a one-segment journey, so every consumer reads one shape and no caller grows its own fallback.
- **Provider / context.** AppProvider supplies session state and a storage-ready flag to the whole tree, giving pages a single place to wait for storage to become available.
- **Guard clause.** requireAdmin is applied at the top of every administrative operation, so authorisation is enforced in one place rather than being scattered.
- **Design tokens.** Every colour, duration, radius and type step is a named CSS custom property. One class name resolves correctly in both appearances without a dark: variant on every element.

CHAPTER FIVE: SYSTEM DESIGN

5.1 Domain Model

Nine entities constitute the domain. The model is normalised in the sense that no fact is stored twice: seat availability is derived from bookings, and passenger counts are derived from the passenger array, so neither can drift out of step with its source.

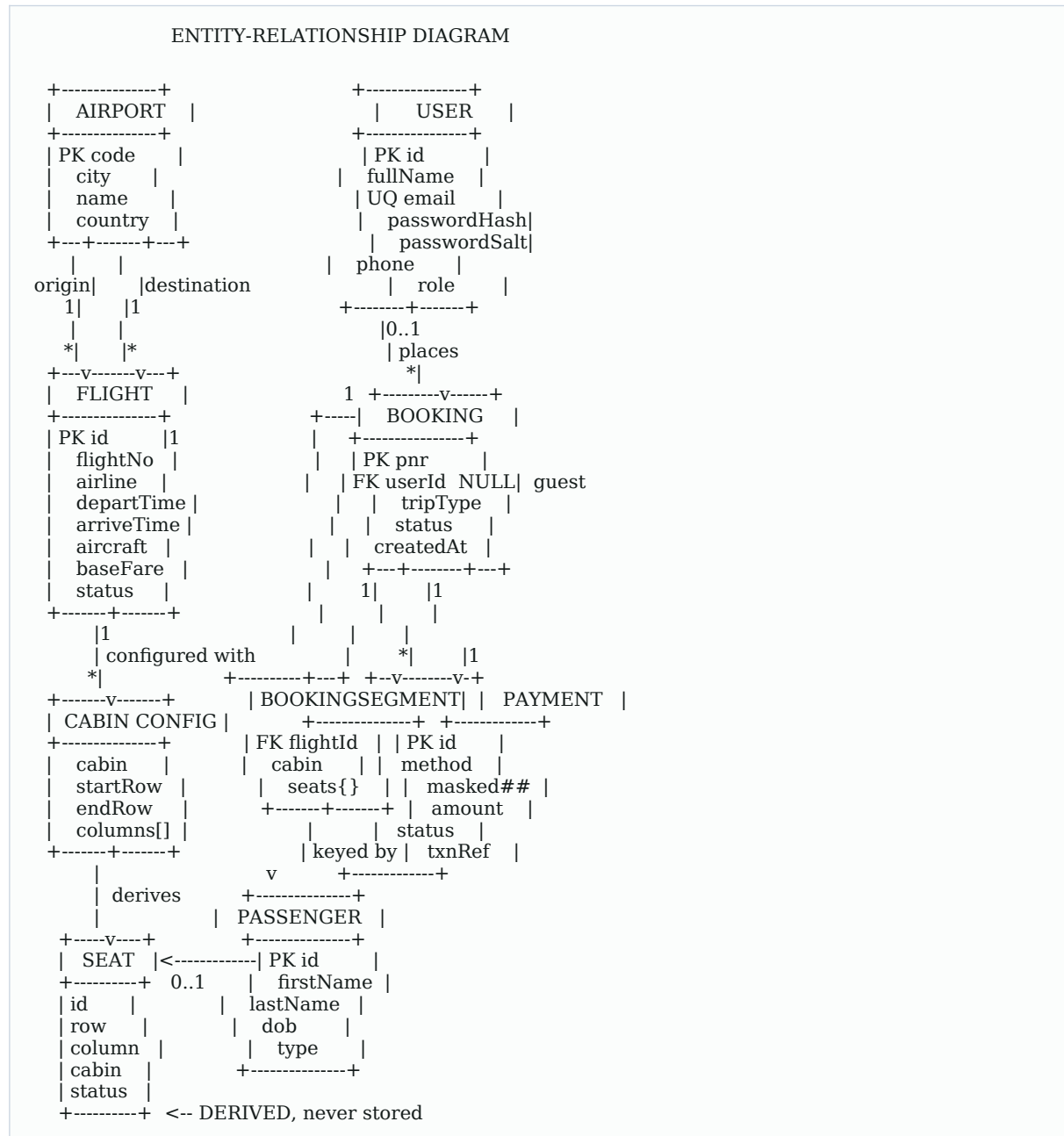


Figure 5.1 Entity-relationship diagram. SEAT is computed from CABIN CONFIG and BOOKINGSEGMENT.

Entity	Key attributes	Notes
Airport	code (PK, IATA), city, name, country	Sixteen airports seeded. The country field

Entity	Key attributes	Notes
		drives the international/domestic decision that determines whether a passport is required.
Flight	id (PK), flightNumber, airline, airlineCode, originCode (FK), destinationCode (FK), departureTime, arrivalTime, durationMinutes, aircraft, baseFare, currency, cabins[], blockedSeats[], status	The composite id combines flight number, date and departure time, guaranteeing uniqueness for a service on a given day.
CabinConfig	cabin, startRow, endRow, columns[]	Embedded in Flight. Defines the physical layout from which the seat map is generated. Six aircraft types are modelled.
Seat	id, row, column, cabin, isWindow, isAisle, isExitRow, status	Derived at run time; never persisted. Status is computed from blockedSeats and the confirmed booking set.
User	id (PK), fullName, email (unique), passwordHash, passwordSalt, phone, role, createdAt	Password material is stored only as a salted, iterated digest. The role attribute drives all authorisation.
Booking	pnr (PK), userId (FK, nullable), tripType, segments[], passengers[], fare, payment, status, contactEmail, contactPhone, createdAt, cancelledAt, refundAmount	The aggregate root. A null userId marks a guest booking, reachable only by reference and surname.
BookingSegment	flightId (FK), cabin, seats (passengerId → seatId)	One per flight on the journey, in flown order. Seats live here rather than on the passenger because the same traveller holds a different seat on each leg.
Passenger	id, title, firstName, lastName, dateOfBirth, gender, passportNumber, type, seatId	Stored inside its booking. seatId is retained for single-segment bookings so a one-flight itinerary reads without indexing into segments.
Payment	id, method, maskedCardNumber, cardHolder, amount, currency, status, transactionReference, paidAt	Only the last four digits of a card are ever retained. The full number never reaches storage.
FareBreakdown	baseFareTotal, cabinSurcharge, seatSelectionFee, taxes, serviceCharge, total	Stored with the booking so the price paid is preserved even if fare rules later change.

Table 5.1 Entity attribute definitions.

5.2 Persistence Design

localStorage is a synchronous, origin-scoped, string-keyed store. It has no schema, no query language, no transactions and no type safety. The persistence design compensates for each of these deficiencies deliberately.

Key	Contents	Approximate size
skyroute:state	The complete application state — schema version, airports, flights, users and bookings — as a single JSON document.	A few hundred KB when seeded
skyroute:session	The currently signed-in principal: id, name, email and role. Never contains credential material.	Under 1 KB
skyroute:guest	A flag marking the current visitor as deliberately booking without an account.	A few bytes
skyroute:booking-draft	An optional in-progress booking, allowing a partially completed wizard to be resumed.	A few KB

Table 5.2 *localStorage* key schema.

Storing the entire state under a single key was a deliberate choice over spreading entities across many keys. *localStorage* offers no transaction across keys, so a multi-key write could be interrupted midway and leave the store internally inconsistent — a booking written but a seat not marked, for instance. A single-key write is atomic at the level the browser exposes: either the whole new document replaces the old one or it does not. Given the size of the dataset, the cost of rewriting it on each mutation is negligible, and it buys a consistency guarantee that would otherwise require a hand-written journal.

Four further defences are built into the persistence layer: every key is namespaced with a *skyroute:* prefix so the application never destroys data belonging to another application on the same origin; every access is guarded by a write probe, because *localStorage* throws in private-browsing mode in some browsers; every read is wrapped so malformed JSON degrades to a documented default rather than propagating an exception into the render tree; and a schema version is stored alongside the data so a future change to the model can trigger a controlled migration rather than misinterpreting old records.

5.3 Process Design

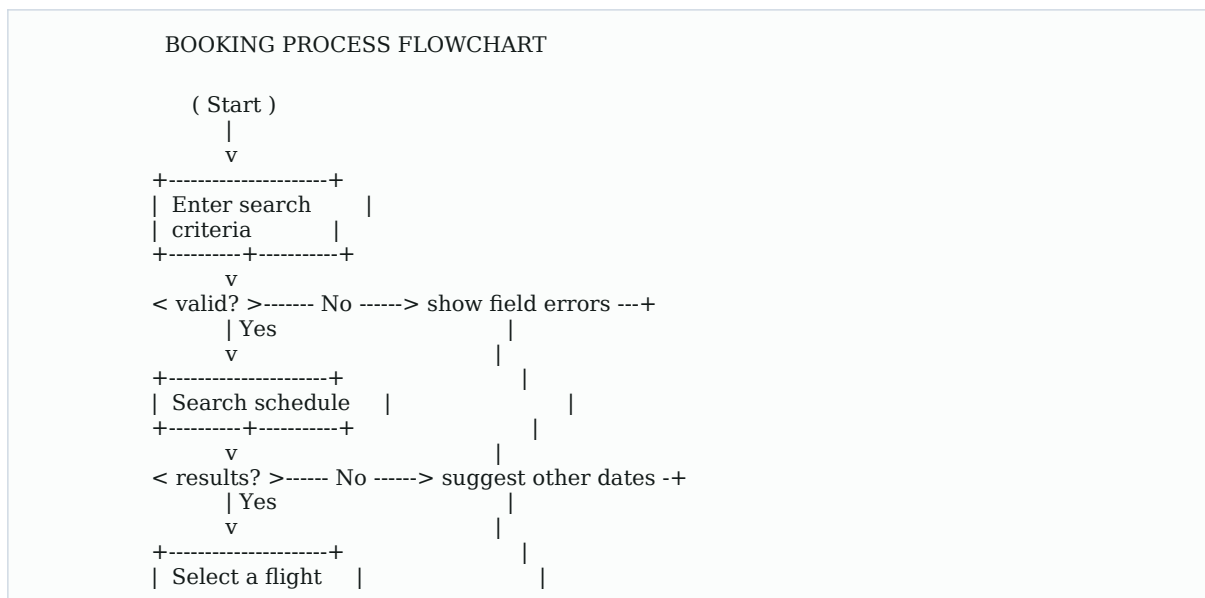
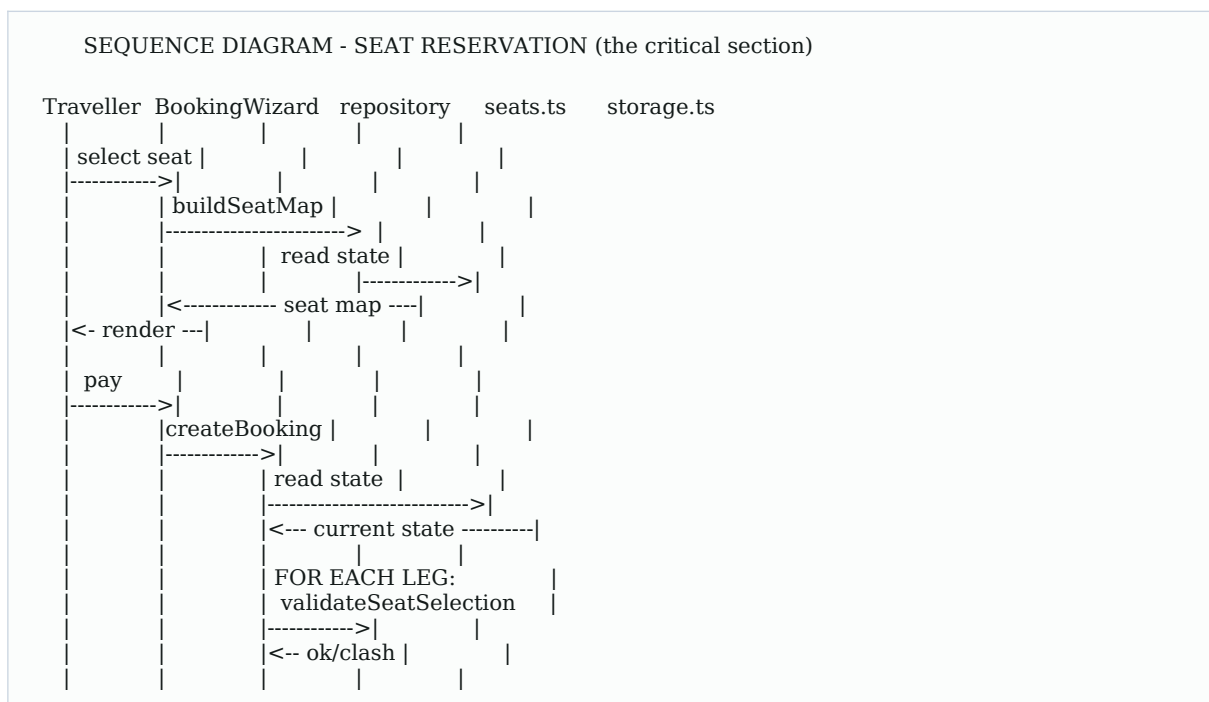




Figure 5.2 Booking process flowchart. The re-verification step is what makes double-selling impossible.



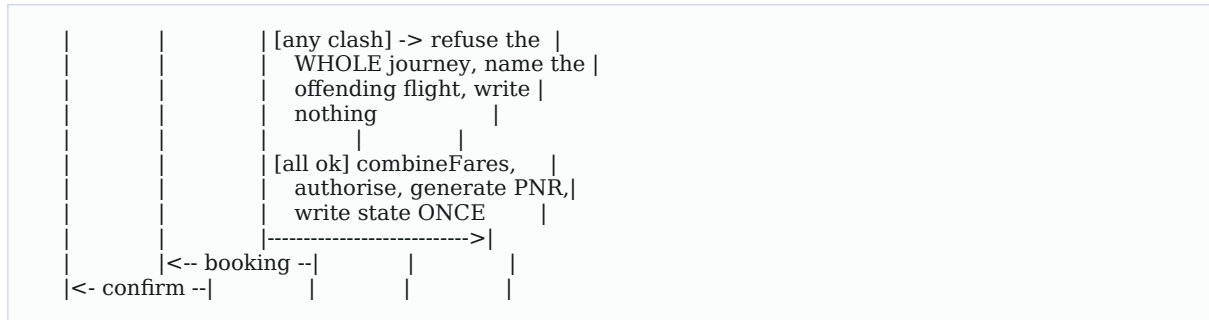


Figure 5.3 Sequence diagram for seat reservation. State is re-read immediately before the write.

5.4 Algorithm Design

5.4.1 The fare engine

Fares are constructed multiplicatively from four independent factors. Each is a pure function of a single input, which makes the whole engine exhaustively testable and makes the effect of any one factor easy to reason about in isolation.

```

farePerPassenger =
  baseFare
  x advancePurchaseFactor(daysToDeparture) // 0.90 -> 1.50
  x demandFactor(cabinLoadFactor) // 1.00 -> 1.35
  x passengerTypeFactor(type) // 1.00 / 0.75 / 0.10
  x cabinFactor(cabin) // 1.00 / 2.60 / 4.20

legTotal =
  SUM farePerPassenger
  + SUM seatSelectionFee // economy only
  + VAT at 7.5% of the above
  + flat service charge of NGN 2,500
  
```

Days before departure	Factor	Rationale
30 or more	0.90	Rewards early commitment; the seat is sold long before it would otherwise be.
14 to 29	1.00	The reference fare against which the others are expressed.
7 to 13	1.15	Demand hardens as departure approaches.
3 to 6	1.32	Late booking, typically less price-sensitive.
Fewer than 3	1.50	Last-minute travel, historically the least price-elastic segment.

Table 5.3 Advance-purchase factor.

Cabin load factor	Factor	Rationale
Below 0.50	1.00	Ample inventory; no scarcity premium.
0.50 to 0.74	1.08	Demand confirmed; a modest premium is applied.

Cabin load factor	Factor	Rationale
0.75 to 0.89	1.20	Scarcity is real; remaining seats are more valuable.
0.90 and above	1.35	Near capacity; the last seats command the highest fare.

Table 5.4 Demand factor derived from cabin load.

Two refinements are worth noting. The itemised quotation separates the economy-equivalent base fare from the cabin uplift, so a passenger booking Business sees what the flight would have cost in Economy and what the upgrade specifically costs them — more informative than a single opaque figure, and requested directly by interview participants.

The second concerns multi-leg journeys. Combining per-leg quotes naively would be wrong twice over: the booking service charge is levied once per booking rather than once per flight, so summing whole quotes would charge a return trip twice for it; and tax is recomputed from the combined taxable base rather than summed from each leg's already-rounded figure, so the total cannot drift a naira away from the stated rate. Both behaviours are asserted by test.

```
function combineFares(fares):
  if only one fare: return it unchanged

  baseFareTotal    <- sum of each leg's baseFareTotal
  cabinSurcharge   <- sum of each leg's cabinSurcharge
  seatSelectionFee <- sum of each leg's seatSelectionFee

  taxable <- baseFareTotal + cabinSurcharge + seatSelectionFee
  taxes   <- round(taxable x VAT_RATE)      // recomputed, not summed

  return { ..., serviceCharge: SERVICE_CHARGE, // once, not per leg
           total: taxable + taxes + SERVICE_CHARGE }
```

5.4.2 Seat map derivation

The seat map is computed, never stored. The algorithm takes a flight and the current booking set and produces the complete map:

```
function buildSeatMap(flight, bookings):
  occupied <- { every seatId held on ANY segment of a CONFIRMED
                booking whose flightId is this flight }
  blocked  <- flight.blockedSeats
  exitRows <- EXIT_ROWS[flight.aircraft]

  for each cabin in flight.cabins:
    for row from cabin.startRow to cabin.endRow:
      for (index, column) in cabin.columns:
        id      <- row + column
        isWindow<- index is first or last in the row
        isAisle <- column borders an aisle break
        status  <- blocked? 'blocked'
                  : occupied? 'occupied'
                  : 'available'
```

```
return seats
```

Complexity is $O(\text{rows} \times \text{columns})$, bounded by the largest aircraft modelled. The important property is not the complexity but the invariant: because availability is a function of the booking record, **there exists no state in which the seat map and the bookings disagree**. Overselling is not prevented by a check; it is prevented by the shape of the data. Requirement FR-14 — cancelled seats return to the pool — therefore needs no implementation at all, on any leg, because occupancy is defined as membership of a confirmed booking.

5.4.3 PNR generation

Record locators are six characters drawn from a 32-character alphabet from which I, O, 0 and 1 have been removed — the precaution real carriers take, because these characters are routinely confused when a reference is read aloud at a check-in desk. The alphabet yields roughly 1.07 billion distinct codes.

```
ALPHABET = 'ABCDEFGHJKLMNPQRSTUVWXYZ23456789' // 32 chars, no I O 0 1

function generateUniquePnr(existing, maxAttempts = 50):
  for attempt in 1..maxAttempts:
    candidate <- 6 characters drawn with crypto.getRandomValues
    if candidate not in existing: return candidate
  // Guaranteed termination: fall back to a timestamp-derived code
  return first2(random) + base36(now).lastFour
```

Randomness is drawn from the Web Crypto API where available, falling back to Math.random only in environments that lack it. The bounded retry with a deterministic fallback means the function cannot loop indefinitely even in the pathological case where every candidate collides.

5.4.4 Refund calculation

The refund policy is a step function of the interval remaining before departure. The service charge is excluded from the refundable base, and the interval is measured from the first departure of the journey rather than any later leg — a return trip cannot be cancelled at full value simply because the return flight is still weeks away.

Time before the first departure	Refund of the refundable amount
168 hours (7 days) or more	90%
72 to 168 hours (3 to 7 days)	70%
24 to 72 hours (1 to 3 days)	50%
Under 24 hours	0% — non-refundable

Table 5.5 Cancellation refund schedule. The NGN 2,500 service charge is never refunded.

5.5 Interface and Appearance Design

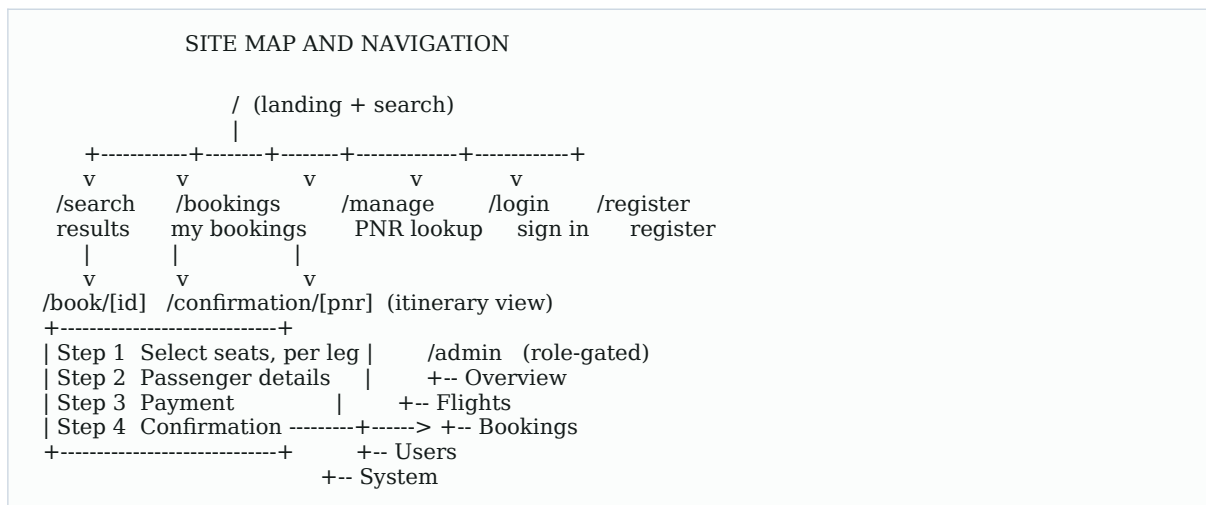


Figure 5.4 Site map and navigation structure.

5.5.1 Design tokens

Every colour, duration, radius and type step is a named CSS custom property declared once in `app/globals.css` and exposed to Tailwind through the theme configuration. One class name therefore resolves correctly in both appearances without a `dark:` variant on every element. The type scale carries its own leading and tracking per step, because those are size-specific: display text needs negative tracking and tight leading while small text needs the opposite, and one global letter-spacing is wrong at one end of the scale or the other.

5.5.2 Colour and contrast

The identity is built on a deep teal. The mark is a disc rather than the near-universal rounded square, which is what makes it findable in a dock or a tab strip, and a disc is the right shape to hang a great-circle route on. The accent value differs between appearances because a colour that reads as deep on white reads as muddy on black.

One hazard in a teal palette needed designing around. Teal and green are neighbours on the wheel, and a confirmed badge has to be unmistakable beside a teal button. The success colour is therefore held at a distinctly warmer, yellower green — a separation of roughly 70 degrees of hue in both appearances — and it never ships without a tick beside it, so the distinction never rests on colour alone.

Token	Light	Dark	Contrast against its surface
--canvas	#e8edeb	#000000	Page ground
--surface	#f5f8f7	#171d1c	Card ground
--ink	#131a19	#f2f6f5	15.8:1 light · 19.3:1 dark
--ink-2	#4d5a57	#9daaa7	6.5:1 light · 7.1:1 dark

Token	Light	Dark	Contrast against its surface
--accent	#0d6b5e	#2fc4ae	5.7:1 light · 9.6:1 dark
--positive	#3d7a24	#7bc44a	Hue 103 / 96 against the accent's 172 / 171
--warn	#a86a12	#ff9f0a	Amber, used with an icon
--danger	#c62f34	#ff453a	Red, used with an icon

Table 5.6 Colour tokens and their measured contrast ratios.

5.5.3 Motion and preference

Motion is used where it explains something. The route on the landing surface draws itself and an aircraft flies along it, which says "a flight from here to there" faster than a sentence does; wizard steps travel in the direction the user is going; feedback on a press lands on pointer-down rather than on release, because that is when the user expects it.

All of it is conditional. The stylesheet carries dedicated blocks for four reader preferences, and reduced motion is treated as "gentler, not absent": the opacity changes that explain what happened stay, and the travel that can cause discomfort goes. The route is shown already drawn rather than drawing, and the aircraft is parked at the destination rather than flown across the viewport.

- **prefers-reduced-motion: reduce** — travel removed, cross-fades kept, scroll-behavior set to auto.
- **prefers-reduced-transparency: reduce** — translucent chrome becomes opaque and backdrop blur is dropped.
- **prefers-contrast: more** — borders strengthen, muted text darkens, materials become solid.
- **prefers-color-scheme** — resolved by next-themes into an explicit class on the root element, so a theme the reader picked is not fought by the operating system preference still matching underneath.

Hover-dependent motion is confined to a (hover: hover) and (pointer: fine) query, because touch fires a false hover on tap and leaves the element stuck in its hover state.

5.6 Security Design

A client-side system cannot offer server-grade security, and it would be dishonest to claim otherwise. What it can do is apply every control that remains meaningful in its context, and state plainly which threats it does not address. The threat model below was constructed before implementation and each control was implemented and then verified by test.

Threat	Control implemented	Residual risk
Password disclosure from stored state	Passwords are stored only as a SHA-256 digest, individually salted per account and iterated 1,000 times. No plain-text password exists anywhere in	An attacker with the device could mount an offline brute-force attack. Iteration raises its cost but does not

Threat	Control implemented	Residual risk
	the system.	eliminate it.
Timing attack on sign-in	Digest comparison is constant-time over the full length, so response time reveals nothing about how many characters matched.	Negligible in this deployment model.
Account enumeration	An unknown email and a wrong password return an identical error message.	None material.
Payment card exposure	The card number is validated and then discarded. Only the last four digits are retained, masked. An automated test asserts the full number never appears in stored state.	None: the data is not present to be exposed.
Privilege escalation	Authorisation is enforced inside the repository by a guard applied to every administrative operation. Hiding a control is treated as a usability measure only.	A user who edits their stored session role could pass the client-side check. Only a server can close this; Section 13.2 describes how.
Cross-user data access	Cancellation verifies that the requesting principal owns the booking, unless that principal holds the administrator role.	Bounded by the same limitation as above.
Guest booking hijack	A guest cancellation is modelled as a distinct requester type and must produce a surname that matches a passenger; a booking that belongs to an account cannot be cancelled by the guest path at all.	Anyone holding both the reference and the surname can cancel, which is the same exposure a printed ticket carries.
Script injection through stored data	All user-supplied text is length-limited and stripped of angle brackets before being written. React additionally escapes all interpolated text.	Low. Two independent controls apply.
Corrupted or tampered stored state	Every read is defensive: malformed JSON, wrong types and missing fields all degrade to documented defaults.	Data loss remains possible, but the application does not crash and cannot be driven into an inconsistent state.
Seat double-allocation between tabs	Availability is re-verified against freshly read state immediately before the write, on every leg.	A theoretical window of a few milliseconds remains. A server transaction would close it.

Table 5.7 Threat model and implemented mitigations.

An honest statement of the security boundary

Everything this system knows is stored on the user's own machine, and the code that checks permissions runs on that same machine. A user with developer tools can therefore read and alter the stored state, including their own role.

This is not a defect in the implementation; it is a property of the deployment model. The controls above are the ones that remain meaningful within that model, and they are implemented properly rather

than performatively.

The moment authentication must be trustworthy, it must move to a server. Section 13.2 specifies exactly that migration, and the architecture was designed from the outset to make it a contained change.

CHAPTER SIX: DEVELOPMENT METHODOLOGY

6.1 Selecting a Methodology

The choice of methodology was made against the specific circumstances of this project: a nineteen-member group in which implementation was carried out by a single developer while the remaining members handled requirements, documentation, testing and review; a fixed academic deadline; requirements understood in outline but not in detail; and a technical approach — client-side-only persistence — whose feasibility could not be assumed and had to be established by building.

Methodology	Characteristics	Assessment
Waterfall	Sequential phases with formal sign-off; requirements frozen before design begins.	Rejected. Several requirements — alternative-date suggestions, guest booking, multi-city journeys, the additional payment methods — emerged only once users handled a working prototype. Waterfall would have discovered them after the design was frozen.
V-Model	Waterfall with each development phase paired to a corresponding test phase.	Rejected as the overall process, but its central discipline was adopted: every requirement is paired with the tests that verify it, expressed as the traceability matrix in Section 9.4.
Spiral	Iterative with formal risk analysis at the start of each cycle.	Rejected. The rigorous risk-analysis apparatus is disproportionate to a one-semester academic project, though risk was reviewed informally at each planning session.
Agile — iterative and incremental	Short iterations, a prioritised backlog, a working increment at the end of each, and a review that feeds the next.	Selected. Delivers something working early, absorbs discovered requirements without disruption, and — critically for a group in which only one member writes code — gives the other eighteen a running system to test, document and critique at the end of every iteration rather than at the end of the project.
Extreme Programming	Pair programming, test-first development, continuous integration, collective ownership.	Not adopted wholesale. Pair programming and collective code ownership presuppose several developers and were therefore inapplicable. Test-first development of the domain layer was adopted and is the practice most visible in the delivered code.

Table 6.1 Comparison of software development methodologies against this project's circumstances.

6.2 Justification for an Iterative Approach

1. **Requirements genuinely evolved.** Guest booking, multi-city journeys and the bank-transfer and wallet payment paths were not in the original specification. They were added because reviewing a working increment made their absence obvious. A methodology that froze the specification before that review would have had to reject them or reopen a completed phase.

2. **Technical risk needed to be retired early.** Whether localStorage could carry a system of this size, and how server rendering would interact with a store that only exists on the client, were open questions. Insisting on a working increment each iteration forced those questions to be answered at the start rather than discovered in the final week.
3. **A single developer is a bottleneck that must be managed.** With implementation concentrated in one person, the risk was that the other eighteen members would have nothing to work against until the end. Short iterations removed that risk: each one produced a running increment that the requirements, documentation, testing and review sub-teams could immediately act on, so their work proceeded in parallel with development rather than after it.
4. **The deadline was fixed and the scope was not.** A prioritised backlog with MoSCoW classification meant that if time ran short, the items dropped would be the ones marked Could rather than whichever happened to be unfinished.
5. **The work is legible in the history.** Development is recorded across 33 commits with conventional prefixes — feat, fix, refactor, style, perf, docs — so the sequence in which capability arrived is auditable rather than asserted.

6.3 Iteration Plan

Iteration	Focus	Principal deliverables
1	Foundation and risk retirement	Project scaffold; domain model in TypeScript; localStorage adapter with probing and corruption handling; schedule generator; first unit tests. The interaction between server rendering and a client-only store investigated and resolved.
2	Search and discovery	Search form with full validation; search algorithm; result cards; sorting, filtering and alternative-date suggestion; fare engine with all four factors and its unit tests.
3	The booking transaction	Seat map generation and the interactive component; four-step wizard; passenger capture with age-band cross-check; simulated card payment with Luhn validation; PNR generation; confirmation and printable itinerary.
4	Accounts, guests and post-booking management	Registration and sign-in; salted iterative password hashing; session handling; My Bookings; cancellation with refund quotation; retrieval by reference and surname; guest mode and guest cancellation.
5	Journeys, payment breadth and administration	Return and multi-city journeys with per-segment seating and combined fares; bank transfer and wallet payment paths; administration console across five modules; authorisation guards; statistics aggregation.
6	Identity, motion and accessibility	Visual identity and the disc mark; design-token system with light and dark appearance; motion design; the four preference media queries; landing surface; final testing and this report.

Table 6.2 Iteration plan and principal deliverables.

6.4 Team Organisation

Implementation was carried out by one member. This was a deliberate decision rather than an accident of circumstance: a codebase of this size, with a domain layer whose correctness depends on invariants that span several modules, is more reliably built by one person holding the whole design in mind than by a large group each holding a fragment of it. The cost of that decision — a single point of failure and a concentration of knowledge — is acknowledged in Section 11.2.1 and addressed in the recommendations.

The remaining eighteen members were organised into five sub-teams covering the work that surrounds the code: establishing what the system had to do, recording how it was built, exercising it as users, and reviewing the result. Each sub-team had a defined remit and a nominated lead, and each worked against the running increment produced at the end of every iteration.

Sub-team	Members	Remit
Development	Ezedimbu Ebube Emmanuel (2022514081) — Team Lead / Sole Developer	Sole responsibility for the software itself: the architecture, the domain model, every module of source code, the automated test suites and the visual identity. All implementation decisions recorded in Chapters Four, Five and Eight are the work of this member.
Coordination & Administration	Uba Thelma Chidinma (2022514029) — Assistant Coordinator / Secretary Nnolum Vivian Chisom (2022514008) — Scheduling & Meeting Records Officer Micheal Ezra Chukwuebuka (2022514105) — Resources & Logistics Officer	Ran the iteration planning sessions and reviews, maintained the backlog and the schedule, kept the minutes and action lists, and managed submission logistics.
Requirements & Research	Anuforo Valentine Uzochukwu (2022514073) — Requirements Gathering Lead Ogbodo Chimdindu Favour (2022514051) — Stakeholder Interview Officer Ugwu Anthony Chukwuemeka (2022514076) — Comparative Systems Researcher Nwachukwu Augustine Ifechukwu (2022514043) — Airline Fare & Policy Researcher	Elicited and specified the functional and non-functional requirements, interviewed prospective users, surveyed the comparable systems reviewed in Chapter Two, and researched the real-world airline fare and cancellation policies on which the pricing rules are modelled.
Documentation & Writing	Anekwe Favour Uchechukwu (2022514181) — Documentation Lead Unachukwu Chimamanda Jessica (2022514050) — Technical Writer — Requirements & Design Ekemezie Chibuikem Azuka (2022514099) — Technical Writer — Implementation & Testing Elobisi Michael Chukwuebuka (2022514042) — Diagrams & Figures Officer	Drafted, structured and edited this technical report from the developer's design notes and code walkthroughs, and prepared the diagrams, tables and figures throughout.

Sub-team	Members	Remit
User Testing & Feedback	Kenechi Richard Michael (2022514179) — User Acceptance Testing Lead Ogheneakoro Ugochukwu Akpewwe (2022514151) — Exploratory Tester — Search & Booking Okeke Isreal Chukwuka (2022514011) — Exploratory Tester — Accounts & Administration Chukwuka Kamsiyochukwu Nelson (2022514185) — Defect Log Recorder	Carried out manual and exploratory testing of each delivered increment as ordinary users, ran the user acceptance test cases in Section 9.6, and recorded and tracked the defects in the log at Section 11.1.
Review & Presentation	Ukohah Onyinye Nkechi (2022514143) — Usability & Accessibility Reviewer Nwoye David Chiagoziem (2022514172) — Editorial & Proofreading Reviewer Iheanyi Valentine Kelechi (2022514117) — Presentation & Demonstration Lead	Reviewed the interface for usability and accessibility, proofread the report, and prepared the demonstration and defence of the system.

Table 6.3 Sub-team composition and responsibilities.

6.5 Definition of Done

A backlog item was considered done only when every one of the following held:

- The acceptance criteria stated on the item are demonstrably met.
- Automated tests cover the new behaviour and the full suite passes.
- TypeScript compiles in strict mode with no errors.
- The behaviour has been exercised by the user testing sub-team against the acceptance criteria and accepted.
- The behaviour has been verified manually in more than one browser.
- Keyboard operation and focus visibility have been checked for any new interactive control.
- Any user-visible change is reflected in the documentation.

6.6 Collaboration Practices

- **Version control.** Git, with short-lived branches merged into the mainline. The commit history uses conventional prefixes so the log reads as a record of capability rather than of activity.
- **Code walkthroughs in place of peer review.** With one developer, conventional peer review of code was not available. Its function was served instead by a walkthrough at each iteration review, in which the developer explained the newly written modules to the documentation sub-team in enough detail for them to write about it. Explaining a design to someone who must restate it in their own words is a demanding test of that design, and it caused more than one module to be reworked before it was accepted.
- **Independent testing.** The most valuable external check was that the people exercising the system were not the person who wrote it. The user testing sub-team worked from the

acceptance criteria and their own curiosity, and found defects — D-02 in particular — that no test written by the author had anticipated.

- **Written conventions.** A style guide covering naming, file organisation and comment policy was written early and followed throughout. The convention adopted — that a comment explains why, never what — is visible across the source and is what made the walkthroughs tractable.
- **Decisions recorded at the time.** Design decisions were written down as they were taken, which is why Chapters Four and Five state the alternatives considered rather than reconstructing them afterwards.

CHAPTER SEVEN: TECHNOLOGIES AND TOOLS

7.1 Programming Language

Language	Strengths	Assessment
JavaScript	Runs natively in every browser; no build step; the largest package ecosystem in existence.	Rejected. Without static types an incorrect property name or a misordered argument is discovered at run time, by a user, rather than at compile time by the author. With a single developer and no second pair of eyes on the code, a compiler that checks every call site is the strongest safety net available.
TypeScript	JavaScript with static types; excellent editor tooling; types serve as executable documentation.	Selected. The domain model becomes a set of enforced contracts, refactoring across a large team is safe, and the discriminated-union result type makes it impossible to read a result without first checking whether it succeeded.
Python (Django / Flask)	Highly readable; excellent libraries; widely taught.	Rejected. Requires a server process, which the no-database, client-only constraint precludes.
Java (Spring Boot)	Strong typing; mature ecosystem; conventional in enterprise systems.	Rejected. Server-side only, and its verbosity would have slowed a six-iteration schedule considerably.
PHP	Ubiquitous in shared hosting; low barrier to entry.	Rejected. Server-side only, and its ecosystem offers less support for the highly interactive interface this project needs.

Table 7.1 Programming language alternatives considered.

TypeScript 5.7 was selected and configured in **strict** mode. Strict mode was not merely switched on and tolerated: it caught a substantial class of defect during development, most notably every path where a flight or a booking might legitimately be absent and the code had assumed it would be present. The nullable `userId` that models a guest booking is enforced by the compiler at every consumer.

7.2 Framework

Framework	Strengths	Assessment
Next.js 16 (App Router)	File-system routing; React Server Components; first-class TypeScript support; production build with automatic code splitting; asset conventions for icons and social cards.	Selected. Provides routing, bundling, optimisation and a production build with no hand-written configuration, letting the team spend its effort on the domain rather than on toolchain assembly.
Plain React with Vite	Lighter; faster development server; fewer conventions to learn.	Rejected. Routing, code splitting and the production pipeline would all have had to be

Framework	Strengths	Assessment
		assembled and maintained by hand.
Angular	Comprehensive and opinionated; strong typing; dependency injection built in.	Rejected. The learning curve is steep and the ceremony disproportionate to the schedule.
Vue.js / Nuxt	Gentle learning curve; excellent documentation.	Rejected on the narrow ground that the team's existing React experience was greater.
Svelte / SvelteKit	Minimal runtime; compiles away the framework; very concise.	Rejected. A smaller ecosystem and less team familiarity outweighed the elegance of the model.

Table 7.2 Framework alternatives considered.

One consequence of choosing Next.js deserves explicit mention. Next.js renders pages on the server by default, and `localStorage` does not exist during server rendering. Every page that reads persisted state must therefore wait for the client to take over, or the server-rendered markup and the first client render will disagree and React will report a hydration mismatch. This was resolved with a single mechanism described in Section 8.5, applied once rather than worked around in every page.

7.3 Persistence Mechanism

Option	Characteristics	Assessment
localStorage	Synchronous; string keys; roughly 5 MB per origin; universally supported; survives browser restart.	Selected — and mandated by the project constraint. Its synchronous API keeps the repository functions simple, and its size limit is comfortable for a dataset of this scale.
sessionStorage	Identical API but cleared when the tab closes.	Rejected. Bookings must survive a browser restart; a reservation that vanishes with the tab is not a reservation.
IndexedDB	Asynchronous; effectively unbounded; supports indexes and transactions.	A stronger technical fit for a larger dataset, and the natural next step if the schedule grew by an order of magnitude. Rejected here because its asynchronous, event-driven API would add significant complexity for a dataset that fits comfortably in <code>localStorage</code> .
Cookies	Sent with every HTTP request; roughly 4 KB per cookie.	Rejected. Far too small, and transmitting booking data on every request would be both wasteful and a privacy concern.
Relational database	ACID transactions; concurrent multi-user access; durable and backed up.	Excluded by the project constraint, but retained as the explicit migration target and modelled for in the architecture.

Table 7.3 Persistence alternatives considered.

7.4 Complete Technology Stack

Concern	Technology	Version	Role in the system
Language	TypeScript	5.7	Static typing across the whole codebase; types as enforced documentation.
Framework	Next.js (App Router)	16.3	Routing, bundling, production build, and the file conventions that generate the favicon, home-screen icon and social card.
UI library	React	19.0	Declarative component model; hooks for state.
Styling	Tailwind CSS	3.4	Utility classes bound to the design tokens declared in globals.css.
Appearance	next-themes	0.4	Resolves the system preference into an explicit class on the root element, so a chosen theme is not fought by the OS preference underneath.
Animation	Framer Motion	13.1	Component-level transitions: the appearance toggle, tabbed showcases.
Animation	GSAP	3.15	Timeline-driven effects on the landing surface.
Rendering	OGL	1.0	A small WebGL library driving the ambient light effect behind the hero.
Scrolling	Lenis	1.3	Inertial smooth scrolling on the landing surface.
Utilities	clsx, tailwind-merge	2.1, 3.6	Conditional and conflict-free class composition.
Persistence	Browser localStorage	Web API	Mandated constraint; synchronous, universally supported, adequate capacity.
Test runner	Vitest	2.1	Fast, TypeScript-native, integrates with the existing build tooling.
Test environment	jsdom	25.0	Supplies a DOM and a working localStorage to Node, so persistence can be integration-tested without a browser.
Linting	ESLint with eslint-config-next	9.39	Enforces conventions and catches React and accessibility defects.
Version control	Git	—	Branching and an auditable history of how capability arrived.
Runtime	Node.js	—	Executes the build tooling and the test suite.

Table 7.4 Complete technology stack.

7.5 A Note on Dependency Discipline

The domain of this system — the fare engine, the seat map, validation, hashing, PNR generation, the icon set and the logo — is written from scratch and lives in the repository. No library supplies any of it. This was a deliberate decision, for three reasons. The assignment is to demonstrate the application of software development principles, and importing a package that solves the interesting problem would defeat that purpose. Every dependency is a maintenance liability and a supply-chain risk. And, most practically, code the team wrote is code the team can explain in a viva.

The libraries that are present serve the presentation layer only — animation, smooth scrolling, appearance switching and class composition. None of them touches a business rule. An audit conducted while preparing this report found that several packages declared in `package.json` are not imported anywhere in the source; these are recorded as a finding in Section 9.7 and their removal is recommended in Section 13.3.

CHAPTER EIGHT: IMPLEMENTATION

8.1 Overview

This chapter documents how the design of Chapters Four and Five was realised in code. Extracts are given where the code is the clearest statement of a design decision; each is annotated with the reason it is written the way it is.

Area	Files	Lines	Contents
app/	13	4,481	Nine App Router pages — landing and search, results, the booking wizard, confirmation, my bookings, manage booking, sign in, register, administration — plus the root layout, the global stylesheet and the generators for the home-screen icon and social card.
components/	26	5,532	The application components (provider, navigation, search form, flight card, seat map, passenger and payment forms, fare summary, itinerary), the brand mark, the 56-icon set, and the landing-surface components.
lib/	11	2,304	The application, domain and persistence layers: types, storage, repository, pricing, seats, validation, auth, ids, seed, format.
tests/	6	1,826	Five suites and shared fixtures: pricing, seats, validation, auth and repository.
design/	—	—	Logo vectors, the icon contact sheet, and the tooling that regenerates them.

Table 8.1 Source file inventory. Totals exclude configuration and generated assets.

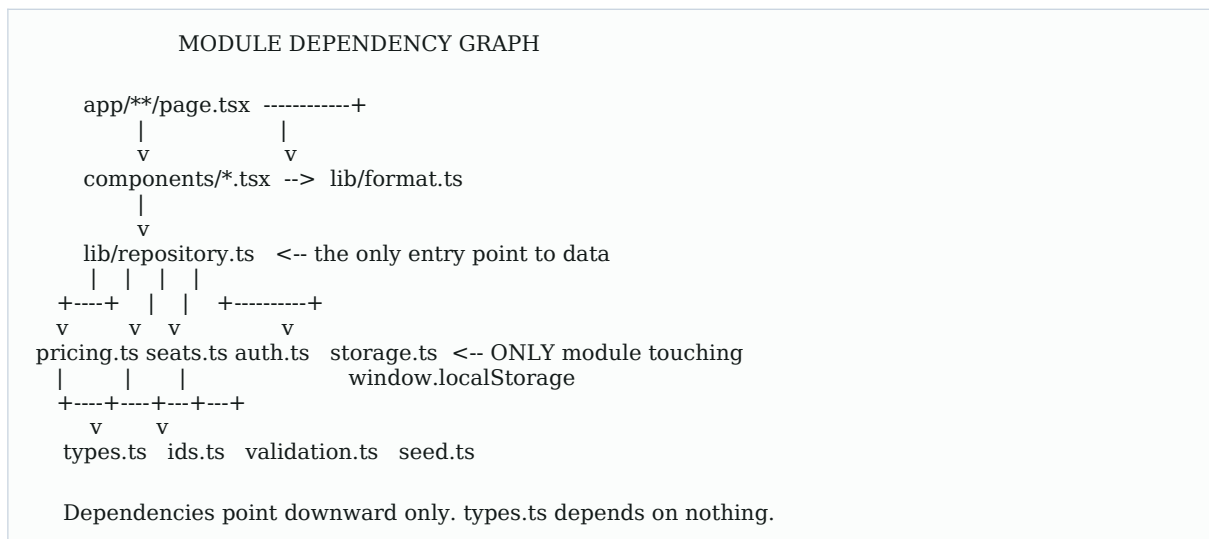


Figure 8.1 Module dependency graph. No arrow points upward.

8.2 The Persistence Layer

The persistence layer is deliberately the smallest module in the system, at 102 lines. Its entire job is to convert between typed objects and the string-keyed store, and to absorb every way in which that store can fail.

```
export const STORAGE_PREFIX = "skyroute:";
export const SCHEMA_VERSION = 1;

/** True when running in a browser with a usable localStorage. */
export function isStorageAvailable(): boolean {
  if (typeof window === "undefined") return false; // server render
  try {
    const probe = `${STORAGE_PREFIX}__probe__`;
    window.localStorage.setItem(probe, "1");
    window.localStorage.removeItem(probe);
    return true;
  } catch {
    return false; // private browsing, or quota exhausted
  }
}

/** Read and parse a namespaced key, returning `fallback` on any failure. */
export function readItem<T>(key: string, fallback: T): T {
  if (!isStorageAvailable()) return fallback;
  try {
    const raw = window.localStorage.getItem(STORAGE_PREFIX + key);
    if (raw === null) return fallback;
    return JSON.parse(raw) as T;
  } catch {
    return fallback; // corrupted or hand-edited value
  }
}
```

Three details carry weight. The guard on `window` is what allows the module to be imported into a page that Next.js renders on the server without throwing. The write probe, rather than a simple feature test, is necessary because in some browsers `localStorage` exists as an object but throws on write in private-browsing mode. And the catch around `JSON.parse` is not defensive pessimism: a user can open developer tools and put arbitrary text under that key at any time, so malformed input is an expected condition rather than an exceptional one.

8.3 The Domain Layer

8.3.1 The fare engine

Every function in the fare engine is pure: the same inputs always produce the same output, and nothing outside the function is read or modified. This is what allows the pricing suite to run in tens of milliseconds with no setup and no mocking.

```
export function advancePurchaseFactor(daysToDeparture: number): number {
  if (daysToDeparture >= 30) return 0.9;
  if (daysToDeparture >= 14) return 1.0;
  if (daysToDeparture >= 7) return 1.15;
  if (daysToDeparture >= 3) return 1.32;
  return 1.5;
}
```

```
export function demandFactor(load: number): number {
  const clamped = Math.min(Math.max(load, 0), 1); // never extrapolate
  if (clamped < 0.5) return 1.0;
  if (clamped < 0.75) return 1.08;
  if (clamped < 0.9) return 1.2;
  return 1.35;
}
```

The clamp in `demandFactor` is a small but real safeguard. A load factor outside the range 0 to 1 would indicate a defect elsewhere, but clamping ensures that such a defect produces a merely wrong price rather than an absurd one — and the corresponding test asserts the clamping behaviour explicitly.

8.3.2 Seat map derivation

The seat map is the clearest illustration of the principle that a fact should be stored once. Availability is not a stored attribute of a seat; it is computed from the booking record every time it is needed. The filter on booking status is what makes cancellation release seats with no code of its own.

```
/** Only CONFIRMED bookings hold seats - cancelling releases them free. */
export function getOccupiedSeatIds(flightId, bookings): Set<string> {
  const occupied = new Set<string>();

  bookings
    .filter((b) => b.status === "confirmed")
    .forEach((booking) => {
      // Every leg is searched, not just the first: a seat held on the
      // return flight of a round trip must be just as unavailable.
      bookingSegments(booking)
        .filter((segment) => segment.flightId === flightId)
        .forEach((segment) => {
          Object.values(segment.seats).forEach((seatId) => {
            if (seatId) occupied.add(seatId);
          });
        });
    });

  return occupied;
}
```

8.4 The Application Layer

`createBooking` is the most important function in the system, and where the transaction boundary is drawn. Its structure — validate everything against freshly read state, then write once — is the mechanism that makes the booking transaction safe across an arbitrary number of legs.

```
export function createBooking(input, now = new Date()): Result<Booking> {
  const state = loadState(); // (1) fresh read

  // (2) validate EVERY leg against THAT state before writing anything
  for (const leg of input.legs) {
    const flight = state.flights.find((f) => f.id === leg.flightId);

    if (!flight) return fail("not found", leg);
    if (flight.status === "cancelled") return fail("cancelled", leg);
    if (departed(flight, now)) return fail("departed", leg);
    if (!availableCabins(flight).includes(leg.cabin))
      return fail("no such cabin", leg);
  }
}
```

```

// (3) THE CRITICAL CHECK - another tab may have taken these seats
//   while this user was filling in the payment form.
const check = validateSeatSelection(
  flight, state.bookings, seatIdsOf(leg), leg.cabin);

//   Once there is more than one flight, saying "a seat is gone"
//   is not enough - the traveller has to be told WHICH one.
if (!check.valid) return fail(check.message, leg);
}

// (4) price each leg, then combine - service charge once, tax recomputed
const fare = combineFares(input.legs.map((leg) => quoteFor(leg, state)));

// (5) authorise payment BEFORE anything is written
if (!paymentSucceeded) return fail("Payment failed. No seats reserved.");

// (6) unique locator, then ONE atomic write of the whole journey
const pnr = generateUniquePnr(state.bookings.map((b) => b.pnr));
state.bookings.push({ pnr, segments, passengers, fare, ... });

if (!saveState(state)) return fail("Booking could not be saved.");
return { ok: true, data: booking };
}

```

1. **Nothing is trusted from the caller.** The interface has already validated the seats, but the state it validated against may be seconds old. The function re-reads and re-validates, because it is the last line of defence.
2. **The journey is atomic.** Every leg is validated before any is written. A four-city itinerary whose third leg has lost a seat writes nothing at all, rather than booking two legs and failing.
3. **Failures are values, not exceptions.** The caller receives a discriminated union and TypeScript will not permit it to read `result.data` without first narrowing on `result.ok`, so a failure cannot be accidentally ignored.
4. **The clock is injectable.** The `now` parameter defaults to the current time but can be supplied by a test, which is what allows departed flights, advance-purchase pricing and refund bands to be tested deterministically.

8.4.1 Authorisation, and the guest case

Authorisation is implemented once, in the application layer, and applied as a guard at the top of every administrative operation. This placement is a security decision: a check performed in a React component can be bypassed by anyone who can call the underlying function, whereas a check inside the function cannot.

Cancellation has three legitimate requesters — the owning account, an administrator, and a guest who can produce a surname from the booking — and the requester is modelled as a discriminated union rather than an optional surname parameter, so a caller cannot quietly pass `neither` and land in an unguarded branch.

```

export type CancelRequester =
  | { kind: "account"; user: SessionUser }
  | { kind: "guest"; surname: string };

```

```
// A guest may cancel ONLY a booking that carries no account, and only
// with a surname that appears on it. A booking that belongs to someone's
// account is not reachable down this path at all.
```

A second guard refuses deletion of a flight that carries confirmed bookings — the job a relational database would do with a foreign-key constraint. Without a database, integrity constraints have to be written by hand, and this is a concrete illustration of what the no-database constraint actually costs.

8.5 The Presentation Layer

8.5.1 Server rendering and the hydration problem

The single most instructive implementation obstacle was reconciling server rendering with client-only storage. The resolution is one flag in the application context, and every page observes the same discipline:

```
const { ready } = useApp();

// Server render and first client render agree: both show the spinner.
if (!ready) return <Spinner label="Loading the schedule" />;

// Only now is it safe to read persisted state.
const results = searchFlights(criteria);
```

Because `ready` is false during server rendering and remains false on the very first client render, the two renders produce identical markup and React hydrates cleanly. The flag flips only in an effect, which by definition runs on the client after mount.

8.5.2 Explanatory motion

The landing surface draws the route and then flies an aircraft along it. Both halves are gated in CSS rather than in JavaScript, so a reader who has asked for reduced motion gets the finished line and no flight, and a browser without CSS motion paths gets no aircraft at all rather than one parked in a corner.

```
/* pathLength="1" on the element makes the dash maths resolution-free. */
.route-line { stroke-dasharray: 1; animation: draw-path ...; }

/* Hidden unless the browser can actually place it on the path. */
.route-plane { display: none; }

@supports (offset-path: path("M 0 0 L 1 1")) {
  .route-plane { display: block; offset-rotate: auto; animation: fly-path; }
}

@media (prefers-reduced-motion: reduce) {
  /* Show the finished route rather than drawing it, and park the
   aircraft at the destination instead of flying it across. */
  .route-line { animation: none; stroke-dashoffset: 0; }
  .route-plane { animation: none; offset-distance: 100%; }
}
```

8.5.3 The appearance system

Appearance is resolved by next-themes into an explicit class on the root element rather than being left to a prefers-color-scheme media query. The reason is a real defect the media-query approach produces: a reader who explicitly selects light while their operating system is set to dark finds the OS-level query still matching underneath and fighting their choice. Resolving the preference into a class on first paint removes the conflict and also removes the flash of the wrong appearance on load.

8.6 Development Practices Observed

- **Test-first for the domain layer.** Pricing, seats, validation and auth were specified as tests before implementation. Writing the assertions first repeatedly clarified the specification — the exit-row fee, for instance, was under-specified until a test forced the question.
- **Comments explain why, not what.** Code that restates itself in a comment is noise. Comments in this codebase record the reason for a decision that is not obvious from the code — why the comparison has no early exit, why the state is stored under one key, why the theme is a class rather than a media query.
- **Injectable clock.** Every date-sensitive function accepts an optional now parameter, which is why time-dependent behaviour can be tested without manipulating the system clock.
- **Errors written for the person reading them.** Every message states what went wrong and what to do about it. "This flight has confirmed bookings. Set its status to cancelled instead of deleting it" is more useful than "Operation failed".

CHAPTER NINE: TESTING

9.1 Testing Strategy

Testing followed a layered strategy, each level answering a different question and catching a class of defect the others cannot.

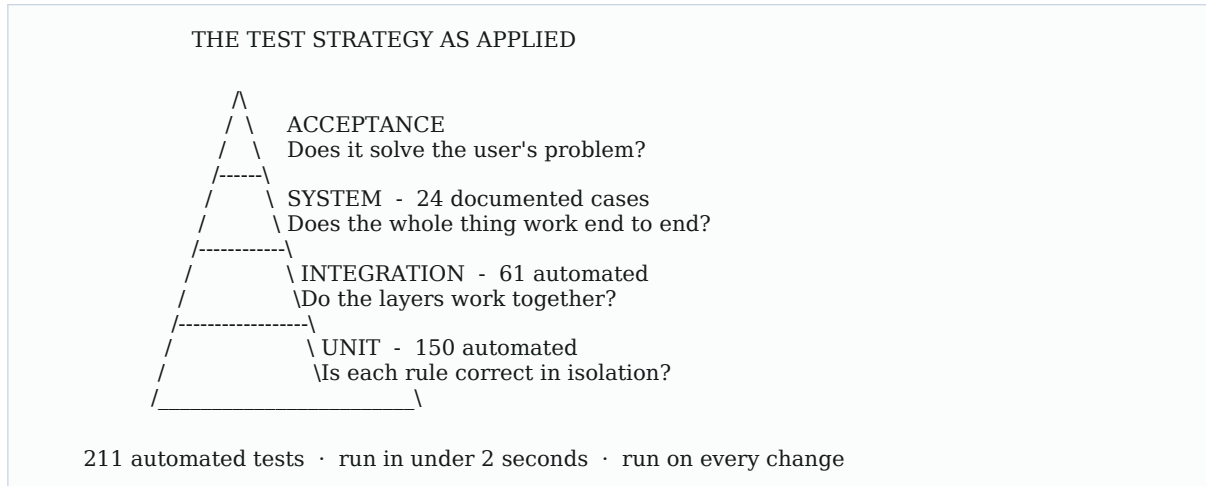


Figure 9.1 Test strategy. The base is broad, fast and automated; the apex is narrow and human.

9.1.1 Test design techniques

- **Equivalence partitioning.** Inputs were divided into classes expected to behave alike — a valid Nigerian phone number, one too short, one with a disallowed prefix — and one representative of each was tested.
- **Boundary value analysis.** Defects cluster at boundaries, so the pricing bands were tested at exactly 30, 29, 14, 13, 7, 6, 3 and 2 days, and the refund bands at exactly 168, 72 and 24 hours.
- **Decision table testing.** The seat-fee rules depend on four booleans (cabin, exit row, window, aisle); the combinations were enumerated and each asserted.
- **State transition testing.** A booking moves confirmed to cancelled and a flight moves scheduled to delayed to cancelled. Both permitted and forbidden transitions were tested, including cancelling an already-cancelled booking.
- **Negative testing.** A large proportion of the suite asserts that something is correctly refused rather than correctly performed. This proportion is deliberate: in a transactional system, refusing the wrong thing matters as much as performing the right one.

9.2 Automated Test Suite

Suite	Tests	Level	Coverage
validation.test.ts	74	Unit	Email, Nigerian phone, name and passport patterns; the four password rules; search rules including the infant-to-adult ratio and the party-size cap; age-band derivation on the travel date; Luhn validation with known-good and known-bad cards; brand

Suite	Tests	Level	Coverage
			detection; expiry; the transfer and wallet paths that must not demand a card number; text sanitisation; PNR format, alphabet and collision rate.
repository.test.ts	61	Integration	Registration and duplicate rejection; sign-in including identical messages for both failure modes; search filtering by route, date, status, departure and capacity; the complete booking transaction across all three payment methods; double-booking refusal; declined-payment rollback; card masking in stored state; cancellation, refunds and seat release; ownership enforcement; the guest cancellation rules; administrative authorisation; deletion refusal for booked flights; statistics aggregation; multi-leg journeys; recovery from corrupted storage.
pricing.test.ts	30	Unit	Advance-purchase and demand factors at every band boundary; passenger-type and cabin multipliers; seat fees by seat characteristic; fare itemisation; VAT; the once-per-booking service charge; refund bands; monotonicity of the pricing curve.
seats.test.ts	25	Unit	Seat generation across three aircraft configurations; window, aisle, middle and exit-row classification; blocked and occupied marking; release on cancellation; seats held on any segment rather than only the first; isolation between flights; load factor; conflict detection including duplicates, wrong cabin and non-existent seats.
auth.test.ts	21	Unit	Salt uniqueness across 200 draws; hash determinism; distinct hashes under distinct salts; constant-time comparison; email normalisation; absence of plain-text passwords in the serialised record; session object contains no credential material.

Table 9.1 Automated test suite summary. Total: 211 tests.

```

$ npm test

RUN v2.1.8

OK tests/validation.test.ts (74 tests)
OK tests/repository.test.ts (61 tests)
OK tests/pricing.test.ts (30 tests)
OK tests/seats.test.ts (25 tests)
OK tests/auth.test.ts (21 tests)

Test Files 5 passed (5)
Tests 211 passed (211)
Duration 1.45s

```

Figure 9.2 Output of the automated suite. All 211 tests pass.

9.3 Illustrative Test Cases

Three tests are quoted because they verify the properties on which the correctness of the system most depends.

```
// The property the entire seat design exists to guarantee.
it("refuses to sell a seat twice", () => {
  expect(createBooking(bookingInput(["20A"])).ok).toBe(true);
  const second = createBooking(bookingInput(["20A"]));

  expect(second.ok).toBe(false);
  expect(loadState().bookings).toHaveLength(1); // nothing was written
});

// A failed payment must leave the system exactly as it found it.
it("writes nothing when the simulated payment is declined", () => {
  const result = createBooking({ ...input, forceFailure: true });

  expect(result.ok).toBe(false);
  expect(loadState().bookings).toHaveLength(0); // no partial record
});

// A journey is all-or-nothing, however many legs it has.
it("refuses the whole journey when any single leg is unbookable", () => {
  // Second leg's seat is taken by someone else first.
  const result = createBooking(twoLegJourney);

  expect(result.ok).toBe(false);
  expect(loadState().bookings).toHaveLength(1); // only the blocker
});
```

9.4 Requirements Traceability Matrix

Every functional requirement is traced to the design section that specifies it, the module that implements it and the tests that verify it. The matrix is the evidence that nothing specified was quietly dropped and nothing implemented is unverified.

Req.	Design	Implementation	Verified by	Status
FR-01	§5.6	repository.register, auth.createUserRecord	repository (3), auth (7), validation (4)	Pass
FR-02	§5.6	repository.login, auth.verifyPassword	repository (3), auth (4)	Pass
FR-03	§5.3	repository.searchFlights	repository (7), validation (7)	Pass
FR-04	§5.4.1	pricing.calculateFare	pricing (30)	Pass
FR-05	§5.4.2	seats.buildSeatMap, SeatMap.tsx	seats (11)	Pass
FR-06	§5.4.2	seats.validateSeatSelection, repository.createBooking	seats (7), repository (2)	Pass
FR-07	§3.2	validation.validatePassenger	validation (9)	Pass
FR-08	§3.2	validation.validatePayment, luhnCheck	validation (15), repository (2)	Pass
FR-09	§5.4.3	ids.generateUniquePnr, repository.createBooking	validation (6), repository (3)	Pass

Req.	Design	Implementation	Verified by	Status
FR-10	§5.5	ItineraryCard.tsx, confirmation route	TC-13, TC-14	Pass
FR-11	§5.5	repository.listBookingsForUser	repository (2), TC-15	Pass
FR-12	§5.5	repository.findBookingByPnrAndSurname	repository (3), TC-16	Pass
FR-13	§5.4.4	repository.cancelBooking, pricing.calculateRefund	repository (10), pricing (5)	Pass
FR-14	§5.4.2	seats.getOccupiedSeatIds (derived)	seats (2), repository (1)	Pass
FR-15	§5.6	repository.requireAdmin	repository (3), TC-20	Pass
FR-16	§5.6	createFlight, updateFlight, deleteFlight	repository (8)	Pass
FR-17	§8.4	repository.listAllBookings, cancelBooking	repository (1), TC-22	Pass
FR-18	§8.4	repository.getStats	repository (3)	Pass
FR-19	§5.4.1	repository.createBooking, pricing.combineFares	repository (8), TC-08	Pass
FR-20	§8.4.1	repository.setGuestMode, cancelBooking	repository (5), TC-17	Pass
FR-21	§5.3	repository.datesWithFlights	TC-06	Pass
FR-22	§5.5	search route sort and filter; autoAssignSeats	TC-05, TC-10	Pass
FR-23	§5.2	repository.ensureSeeded, resetSystem	TC-23, TC-24	Pass
FR-24	§8.5.3	next-themes provider, theme-toggle.tsx	TC-12	Pass

Table 9.2 Requirements traceability matrix. Parenthesised figures are counts of automated tests.

9.5 System Test Cases

System testing was performed manually against the running application, following documented scripts with predetermined expected results.

ID	Test case	Expected result	Status
TC-01	Search a served route on a date with departures	Matching flights listed in departure order with fares and seats remaining	Pass

ID	Test case	Expected result	Status
TC-02	Search with identical origin and destination	Refused; field error states the airports must differ	Pass
TC-03	Search with a departure date in the past	Refused; field error states the date cannot be in the past	Pass
TC-04	Search for two infants with one adult	Refused; error states each infant must be accompanied by an adult	Pass
TC-05	Sort results by price, then filter by airline	Ordering and filtering applied correctly and independently	Pass
TC-06	Search a served route on a date with no departures	Empty state shown with alternative dates that do have availability	Pass
TC-07	Open the seat map for a wide-body flight	Three cabins rendered with correct row and column layout	Pass
TC-08	Build a return journey and book it	Seats chosen per leg; one booking written with two segments; service charge levied once	Pass
TC-09	Attempt to continue with fewer seats than passengers	Advance refused; message states how many seats are required	Pass
TC-10	Use automatic seat assignment for three passengers	Three available seats allocated in the requested cabin	Pass
TC-11	Enter a child date of birth against an adult fare	Field error states the date of birth corresponds to a different fare type	Pass
TC-12	Switch appearance from light to dark and reload	Appearance changes with a crossfade and survives the reload	Pass
TC-13	Complete a booking with a valid test card	Payment authorised; six-character PNR issued; itinerary displayed	Pass
TC-14	Print the itinerary	Print view excludes navigation and controls; itinerary renders cleanly	Pass
TC-15	View My Bookings after booking	Booking listed under Upcoming with correct flights, seats and total	Pass
TC-16	Retrieve a booking by PNR and surname	Itinerary returned; a mismatched surname returns nothing	Pass
TC-17	Book as a guest, then retrieve and cancel it by surname	Booking completes without an account, is absent from every account list, and cancels on a correct surname	Pass
TC-18	Cancel a booking 10 days before departure	Refund quoted at 90% of the refundable amount before confirmation; applied on confirmation	Pass

ID	Test case	Expected result	Status
TC-19	Re-book the seat released by TC-18	Seat available again and bookable	Pass
TC-20	Open the admin console while signed in as a customer	Access denied; administrative data not rendered	Pass
TC-21	Add a flight through the administration console	Flight created and immediately discoverable through search	Pass
TC-22	Delete a flight that has confirmed bookings	Deletion refused; message recommends cancelling the flight instead	Pass
TC-23	Book the same seat in two browser tabs simultaneously	First write succeeds; second refused with the flight and seat named	Pass
TC-24	Reset the system from the administration console	All data cleared; schedule regenerated; user signed out	Pass

Table 9.3 System test cases. 24 executed, 24 passed.

9.6 Security and Accessibility Testing

Test	Method	Result
No plain-text password is stored	Register an account, then inspect the serialised state for the password string	Pass — only a hex digest and its salt are present
Identical passwords produce different hashes	Register two accounts with the same password and compare the digests	Pass — salts differ, so digests differ
Full card number is not persisted	Complete a booking, then search stored state for the card number	Pass — only the masked form is present
Account enumeration is not possible	Compare the error for an unknown email with that for a wrong password	Pass — the messages are identical
Authorisation cannot be bypassed through the interface	Invoke createFlight and deleteFlight directly with a customer principal	Pass — refused at the repository layer
Cross-user cancellation is refused	Attempt to cancel another user's booking	Pass — refused with a permission error
A guest cannot cancel an account's booking	Attempt the guest path against a booking that carries a userId	Pass — refused regardless of surname
Injected markup is neutralised	Enter a script tag as a passenger name and inspect stored and rendered output	Pass — angle brackets stripped on write; React escapes on render

Test	Method	Result
Corrupted storage does not crash the application	Overwrite the state key with malformed JSON and reload	Pass — degrades to an empty state and reseeds

Table 9.4 Security test results.

- **Keyboard-only operation.** The complete journey — search, seat selection, passenger entry, payment, confirmation, cancellation — was executed using only the keyboard. Every control was reachable, focus order followed reading order, and the focus indicator was visible throughout.
- **Contrast.** Every text token was measured against its surface in both appearances. Body and secondary text clear 4.5:1; the accent clears it as both a link colour and a button ground.
- **Preference honouring.** Each of the four preference media queries was exercised by forcing the preference in browser developer tools and confirming the interface responded — travel removed, materials made opaque, borders strengthened, appearance switched.
- **Colour independence.** The interface was reviewed in greyscale to confirm no status is conveyed by colour alone; every status carries an icon as well.
- **Responsive testing.** Pages were exercised from 320 px to 1920 px. No horizontal scrolling or clipped content was observed.

9.7 Static Analysis

TypeScript compiles the whole codebase in strict mode with no errors. ESLint, however, reports findings that remain open at the time of writing. They are recorded here rather than omitted, because a report that claims a clean bill of health its own tooling contradicts is worth less than one that states the position accurately.

Rule	Errors	Warnings	Assessment
react-hooks/set-state-in-effect	10	0	State set inside an effect in the pages that read stored data on mount. It works, and the hydration guard makes it safe, but the idiomatic fix is to derive the value during render or move the read into an event handler. Recommended for the next iteration.
@typescript-eslint/no-explicit-any	4	0	Escape hatches in two landing-surface components that interface with WebGL and animation libraries whose own types are loose. Should be narrowed to unknown and refined.
react-hooks/purity	3	0	Values read during render that the linter cannot prove are pure. Confined to two pages that read persisted state.
react-hooks/use-memo	1	0	A memo whose dependency list the linter cannot verify.
@typescript-eslint/no-	0	6	Leftover bindings in the booking wizard and two

Rule	Errors	Warnings	Assessment
unused-vars			showcase components. Cosmetic; removal is trivial.
Other	0	4	Exhaustive-deps, an ARIA role attribute pairing, and two image-element notices in the icon generators.

Table 9.5 Static analysis findings outstanding: 18 errors and 10 warnings across 28 findings.

None of these findings corresponds to a failing test or an observed defect. They are code-quality signals concentrated in the presentation layer; the domain and application layers, which carry the business rules, are clean. Section 13.3 recommends clearing them as the first task of any continuation.

CHAPTER TEN: RESULTS AND DISCUSSION

10.1 What Was Delivered

The project delivered a complete, working flight booking simulation together with this technical report. The system runs in any current browser, requires no database server and no network connection once loaded, and reproduces the full commercial reservation journey from search through to cancellation, with an administrative console alongside it.

Measure	Value
Application routes	9, plus generators for the favicon, home-screen icon and social card
React components	26
Application, domain and persistence modules	11 files, 2,304 lines
Presentation layer	13 route files (4,481 lines) and 26 components (5,532 lines)
Design tokens and interface styles	943 lines in a single stylesheet
Automated tests	211, all passing, in five suites
Automated suite runtime	Under 2 seconds
System test cases executed	24, all passing
Functional requirements delivered	24 of 24
Non-functional requirements met	18 of 18
Airports, route templates, aircraft types	16, 37, 6
Schedule horizon	21 days, regenerated as departures fall into the past
Icons drawn for the project	56, on one 24-pixel grid
Journey types supported	One-way, return, multi-city
Payment methods simulated	Card, bank transfer, wallet
TypeScript errors in strict mode	0
ESLint findings outstanding	18 errors, 10 warnings (Section 9.7)
Recorded commits	33

Table 10.1 Delivered system by the numbers.

10.2 Evaluation Against the Objectives

Objective (§1.3.2)	Outcome	Evidence
1. Specify traceable requirements	Achieved	24 functional and 18 non-functional requirements, each traced through design, implementation and test in Table 9.2
2. Design a storage-independent layered architecture	Achieved	Four layers with downward-only dependencies; exactly one 102-line module references localStorage
3. Model the problem domain	Achieved	Nine entities with defined attributes and relationships, expressed as TypeScript interfaces enforced at compile time
4. Implement a dynamic fare engine	Achieved	Four independent multiplicative factors; 30 unit tests covering every band boundary; combination logic for multi-leg journeys
5. Implement a derived seat map	Achieved	Availability computed from bookings across all segments; overselling structurally impossible; 25 unit tests
6. Implement the booking transaction with a correct failure path	Achieved	Single-write transaction with pre-write re-validation of every leg; declined payment and any unbookable leg leave storage unchanged, both asserted by test
7. Support account and guest booking, retrieval and cancellation	Achieved	Guest mode with a discriminated requester type; guest bookings excluded from account lists; verified by 5 tests and TC-17
8. Implement an administrative module	Achieved	Five-module console with authorisation enforced in the repository, not the interface
9. Design an accessible visual identity and interface	Achieved	Token-based light and dark appearance, four preference media queries, measured contrast, keyboard-only journey verified
10. Verify through a layered test strategy	Achieved	211 automated tests, 24 system cases, security and accessibility testing (Chapter 9)
11. Document the process	Achieved	This report, with alternatives recorded at each decision point and outstanding findings stated

Table 10.2 Objectives evaluated against outcomes.

10.3 Discussion

10.3.1 The constraint was productive

The most instructive finding is that removing the database made the architecture better rather than worse. A team with a database available can be careless about where data access happens, because the database will still be there tomorrow. A team told that its storage is a 5 MB string-keyed store that a user can edit by hand has a strong incentive to confine that mechanism behind a narrow interface — and confining it is precisely what good layering asks for.

The measurable consequence is that the persistence layer is 102 lines and every other module is unaware of how data is stored. The team can state with confidence that migrating to a relational database would require rewriting one file and making the repository functions asynchronous, because no other module imports the storage adapter.

10.3.2 Deriving state eliminates a class of defect

The decision to compute seat availability from the booking record rather than store it paid for itself repeatedly, and paid twice when multi-leg journeys arrived. Requirement FR-14 — cancelled seats return to the pool — required no implementation at all: it follows from the definition of occupancy. When a booking grew from one flight to several, the only change needed was to search every segment rather than one; had availability been a stored flag, every leg would have needed its own bookkeeping and its own opportunity to drift.

The general principle is worth stating plainly: two representations of the same fact will eventually disagree. Storing one and deriving the other removes the possibility.

10.3.3 Testing the failure paths mattered most

A large share of the automated suite asserts that the system correctly refuses something. This proportion was not planned; it emerged because the failure paths turned out to be where the defects were. The seat double-sell and the multi-leg partial write were both failures to refuse rather than failures to perform.

In a transactional system this is intuitive on reflection. The success path is exercised constantly during development and its defects surface immediately. The failure paths — a declined payment, a seat taken between rendering and submission, a guest attempting to cancel someone's account booking, the third leg of a four-city itinerary losing a seat — are exercised rarely, and only a deliberate test reaches them.

10.3.4 What a database was actually providing

- **Referential integrity.** A foreign key with ON DELETE RESTRICT would have prevented a flight with bookings from being deleted. Without one, the check is an explicit guard, and its absence would have been a silent data-corruption bug.
- **Transaction isolation.** A serialisable transaction would have made the seat double-sell impossible by construction. Re-reading state immediately before the write narrows the window to milliseconds but cannot close it. This is the one requirement the architecture satisfies imperfectly, and it is stated as such.
- **Multi-row atomicity.** Writing a four-segment journey is one document write here, which is atomic by accident of the design rather than by guarantee. A database would make it atomic by contract.
- **Schema enforcement.** A database rejects a row of the wrong shape. localStorage accepts any string, so every read is defensive. TypeScript helps at compile time but offers no guarantee about data arriving from storage at run time.

10.3.5 Accessibility is cheaper when it is not retrofitted

Building the seat map from button elements rather than styled containers cost nothing at the time and delivered keyboard operability and correct announcement without further work. Declaring colours as tokens meant the second appearance was a set of value overrides rather than a second stylesheet. The four preference media queries were written alongside the motion they qualify rather than after it, which is why reduced motion degrades gracefully — the explanatory cross-fades stay, only the travel goes — instead of simply switching animation off.

10.3.6 Where the codebase is uneven

The domain and application layers are the strongest part of the system: pure, heavily tested, and clean under static analysis. The presentation layer is where the outstanding lint findings sit, and where the unused dependencies were found. This is a predictable consequence of how the work was sequenced — the domain was built test-first over several iterations, while parts of the landing surface were built quickly late in the schedule. It is recorded here as an honest characterisation rather than smoothed over.

10.4 Comparison with the Reviewed Systems

Against the four systems reviewed in Chapter Two, SkyRoute is smaller in every commercial dimension and does not compete on inventory, scale or reliability. Where it exceeds them is in transparency: it is the only system among the five whose source can be read, whose pricing algorithm is documented in full, and whose administrative back office is exposed for study. It is also the only one that runs with no server and no database, which is what makes it usable as a teaching artefact where infrastructure is not available.

10.5 Limitations Observed in Practice

The limitations anticipated in Section 1.6 were all encountered, and two behaved differently from expectation.

The storage quota proved a non-issue. The seeded schedule occupies a small fraction of the available space, and the team's projection is that the system would accommodate several thousand bookings before pressure appeared. The quota was expected to be the binding constraint and was not.

The per-browser isolation proved more disruptive than anticipated, chiefly during development. Each team member's browser held an independent dataset, so a defect reproducible on one machine might be invisible on another because the underlying data differed. The team responded by adding the system reset function to the administration console, which regenerates a known state on demand, and thereafter reset before every test session. This turned an obstacle into a useful feature and is why FR-23 exists.

CHAPTER ELEVEN: CHALLENGES ENCOUNTERED

This chapter records the substantive difficulties met during the project and how each was resolved, including the two that were resolved imperfectly.

11.1 Technical Challenges

11.1.1 Server rendering against client-only storage

The problem. Next.js renders pages on the server by default. `localStorage` does not exist there. Every page that read persisted state produced different markup on the server than on the client, and React reported a hydration mismatch on every route.

Approaches tried. Disabling server rendering per component with dynamic imports worked but scattered the workaround across the codebase. Suppressing the hydration warning was rejected outright as concealing a real inconsistency rather than resolving it.

Resolution. A single ready flag in the application context, false during server rendering and on the first client render, becoming true only in an effect after mount. Both renders therefore produce identical markup and hydration succeeds. Every page observes the same discipline.

Lesson. A framework-level constraint deserves a framework-level answer applied once, not a workaround repeated in every file that hits it.

11.1.2 Concurrent seat selection

The problem. Discovered during exploratory testing with two tabs open: both could render the same available seat, and both could book it. The seat map was validated against state held in component memory, which by the time of submission could be seconds out of date.

Resolution. Validation was moved inside the transaction. `createBooking` re-reads state from storage and re-validates every requested seat against it immediately before writing. If a conflict is found the booking is refused and the seats are named.

Honest assessment. This narrows the window to the few milliseconds between the read and the write; it does not eliminate it. A database transaction at serialisable isolation would. This is the single requirement the architecture satisfies imperfectly, and the limitation is stated rather than glossed over.

11.1.3 Growing a booking from one flight to several

The problem. Return and multi-city journeys arrived after the single-flight booking was already working and already had bookings stored against it. A passenger holds a different seat on each leg, so the seat could no longer live on the passenger; but existing records had it there.

Resolution. A segments array was introduced as the authoritative shape, and a single function, `bookingSegments()`, widens an older single-flight booking into a one-segment journey. Every

consumer reads one shape and none has to know which era a booking came from. Without that function, each consumer would have grown its own fallback and they would have drifted apart.

Lesson. When a data shape must change under existing records, put the compatibility in one place and make it the only place that knows there was ever more than one shape.

11.1.4 Fare arithmetic across legs

The problem. The first multi-leg implementation summed whole per-leg quotes. That charged the once-per-booking service fee once per flight, and produced a total that could sit a naira away from the stated tax rate because each leg's tax had already been rounded before being added.

Resolution. `combineFares` sums only the pre-tax components, then recomputes tax from the combined base and adds the service charge exactly once. Two tests now assert each behaviour directly.

11.1.5 Naming the failure in a multi-leg booking

The problem. When a single-flight booking failed because a seat had gone, "that seat is no longer available" was sufficient. On a four-city itinerary it is not: the traveller cannot act on it without knowing which flight.

Resolution. The failure message names the offending flight once there is more than one leg, and a test asserts that it does.

11.1.6 An appearance the operating system fought

The problem. The first implementation drove dark mode from a `prefers-color-scheme` media query. A reader who explicitly chose light while their operating system was set to dark found the query still matching underneath and overriding their choice, and every load flashed the wrong appearance before settling.

Resolution. The preference is resolved into an explicit class on the root element on first paint. The media query no longer participates, so an explicit choice wins and there is no flash.

11.2 Process and Team Challenges

11.2.1 A single developer as a bottleneck

The problem. Concentrating all implementation in one member produced a coherent codebase, but it also made that member a bottleneck and a single point of failure. Any week in which the developer was unavailable was a week in which no new increment appeared, and the other sub-teams — whose work depends on having something to test, document and review — were left idle. It also concentrated knowledge of the system in one person, which is precisely the condition a group project is meant to avoid.

Resolution. Partial, and worth stating plainly. Three measures reduced the impact without removing the cause. The backlog was kept deep enough that the non-development sub-teams always had

accepted increments still to work through. The iteration walkthroughs transferred design knowledge deliberately rather than incidentally, so the documentation sub-team could describe modules they had not written. And the report itself became the mechanism of knowledge transfer: Chapters Four, Five and Eight exist partly so that the design does not live only in one head. The underlying constraint — one developer — was not resolved and is the first item in Section 13.3.

11.2.2 Uneven attention across the codebase

The problem. The domain layer was built test-first and worked over repeatedly. Parts of the landing surface were built quickly and late, and that is exactly where the outstanding static-analysis findings and the unused dependencies are concentrated.

Resolution. Partial. The findings are documented in Section 9.7 rather than hidden, and clearing them is a recommendation in Section 13.3. The honest position is that the care applied to the domain layer was not applied with equal rigour to the presentation layer under schedule pressure, and that a second developer reviewing the work would very likely have caught it.

11.2.3 Estimating unfamiliar work

The problem. Early estimates were substantially wrong, because nothing comparable had been built before. The seat map in particular took several times its estimate, and because a single developer has no capacity to absorb an overrun by reassigning work, every overrun moved the whole schedule.

Resolution. Estimation improved as reference points accumulated. The practical adjustment was to spike genuinely unfamiliar work — a short timeboxed investigation — before estimating it, rather than guessing, and to keep the MoSCoW classification current so that an overrun consumed a Could-have rather than a Must-have.

11.3 Defect Log

ID	Defect	Severity	Root cause	Resolution
D-01	Hydration mismatch on every page reading stored state	High	localStorage is undefined during server rendering, so server and client markup diverged	A single ready flag in the application context; every page renders a loading state until the client has mounted
D-02	A seat could be sold twice with two tabs open	Critical	Availability was validated against state held in component memory, not against current storage	State is re-read and re-validated inside createBooking immediately before the write
D-03	Service charge multiplied by the number of legs	Medium	Whole per-leg fare quotes were summed, including the once-per-booking component	combineFares sums only pre-tax components and adds the charge once; regression test added
D-04	Total drifted from the stated tax rate on multi-leg	Low	Each leg's tax was rounded before being summed	Tax recomputed from the combined taxable base;

ID	Defect	Severity	Root cause	Resolution
	journeys			regression test added
D-05	Seats on a later leg were not treated as occupied	High	Occupancy searched only the first flight of a booking	Every segment is searched; regression test added
D-06	Cancelling a return trip used the wrong departure for the refund band	Medium	The refund was computed from whichever flight was found first	The band is computed from the first departure of the journey; test added
D-07	Guest bookings appeared in an account's booking list	High	Bookings were matched on a userId that was undefined rather than explicitly null	userId is explicitly nullable and guest bookings are filtered out; test added
D-08	Explicit theme choice overridden by the operating system	Medium	Dark mode was driven by a media query that kept matching underneath	Preference resolved into a class on the root element on first paint
D-09	Bank transfer and wallet payments demanded card details	Medium	Payment validation assumed a card in every path	Validation branches on the chosen method; two tests added

Table 11.1 Defect log. All listed defects were resolved and each carries a regression test.

D-02 deserves particular comment because it most shaped the final architecture. It was found during exploratory testing with two browser tabs open — a scenario no scripted test had covered. The fix, re-reading state inside the transaction rather than trusting the state the caller was holding, is now the central structural feature of `createBooking`. It is a good illustration of why exploratory testing earns its place alongside a scripted suite.

CHAPTER TWELVE: CONCLUSION

12.1 Summary

This project set out to design, develop, test and document a functional flight booking simulation that demonstrates the practical application of software engineering principles under a deliberate constraint: no database, and all state persisted in the browser.

That aim was achieved. The delivered system reproduces the complete commercial reservation journey — searching a 21-day schedule across sixteen airports, building one-way, return or multi-city journeys, comparing dynamically priced fares, selecting a seat per leg from an aircraft-specific map, entering passenger details, paying by one of three simulated methods with or without an account, receiving a Passenger Name Record, retrieving the booking later by locator and surname, and cancelling it with an automatically computed refund — alongside a five-module administrative console.

All 24 functional requirements and all 18 non-functional requirements were delivered and verified. The system passes 211 automated tests and 24 documented system test cases, and compiles with no TypeScript errors in strict mode. Static analysis findings that remain open are documented in Section 9.7 rather than omitted.

12.2 Contributions

1. **A demonstration that architectural discipline absorbs infrastructural constraint.** Confining the storage mechanism to a single 102-line module allowed an unusual persistence choice to be adopted without contaminating any business rule. The migration to a conventional database remains a contained change.
2. **A documented, unit-tested fare engine.** Commercial pricing algorithms are closed. This project states its algorithm in full — four independent multiplicative factors with published bands, plus the combination rules for multi-leg journeys — and verifies it with 30 tests at every boundary.
3. **A worked demonstration that deriving state eliminates a class of defect.** Computing seat availability from the booking record makes overselling structurally impossible, discharges an entire requirement without code, and absorbed the change from one flight per booking to many with a one-line adjustment.
4. **An accessible interface built from tokens rather than retrofitted.** A single token layer carries two appearances, and four reader preferences are honoured by dedicated stylesheet blocks written alongside the effects they qualify.
5. **A candid account of the limitations.** The problems the architecture does not fully solve are named and paired with the specific change that would solve them, and the outstanding code-quality findings are published rather than suppressed.

12.3 Lessons Learned

- **A constraint applied early shapes an architecture; a constraint discovered late corrupts one.** Knowing from the first iteration that storage would be replaced is what produced a design in which it can be.
- **Two representations of the same fact will eventually disagree.** Store one, derive the other.
- **Failure paths are where the defects live.** The success path is exercised constantly during development; the refusals are exercised only by a test that deliberately reaches them.
- **When a data shape must change, put the compatibility in one function.** Otherwise every consumer grows its own fallback and they drift apart.
- **Accessibility written alongside the feature costs almost nothing.** Written afterwards, it is a rewrite.
- **Review discipline has to be uniform to be real.** The parts of this codebase that received it are demonstrably better than the parts that did not, and the difference is visible in the static-analysis report.

12.4 Concluding Remarks

The question posed in Section 1.2 was whether a realistic flight booking system — with multi-leg journeys, seat inventory, dynamic pricing, concurrency control, authentication, authorisation and administration — could be built entirely on the client using browser `localStorage`, and what the architecture would have to look like for that to be feasible.

The answer is that it can, and that the architecture has to isolate the storage mechanism completely. Every difficulty the constraint created was absorbed by that isolation, and the problems it could not solve — trustworthy authorisation and transactional isolation — are precisely the two that require a server. That these are the residual problems, and that they are resolved by the same single change, is itself the strongest evidence that the layering did its job.

For a group of nineteen students, the value of the exercise lay less in the artefact than in what building it required: specifying before building, deciding deliberately rather than by default, testing the paths that are easy to neglect, and being precise about what the system does not do. It also demonstrated that a project of this kind is not only code. The requirements that were elicited, the users who exercised each increment and found what its author could not, the record that makes the design legible to a reader who never saw it being written — each of those is load-bearing. Those habits transfer to any system, on any stack, with or without a database.

CHAPTER THIRTEEN: RECOMMENDATIONS

13.1 Recommendations for Practice

1. **Isolate the storage mechanism behind one module from the first commit.** The cost is a few hours; the return is the ability to change a decision that would otherwise be permanent.
2. **Derive rather than duplicate.** Where a fact can be computed from another fact, compute it. Every duplicated fact is a future inconsistency with a defect attached.
3. **Enforce authorisation where the work happens, not where the control is.** Hiding a button is a usability decision. Refusing the operation is the security decision, and only one of the two is enforceable.
4. **Test the refusals.** Budget as much test effort for what the system must reject as for what it must accept.
5. **Model mutually exclusive callers as a union, not as optional parameters.** The guest-versus-account cancellation is safe because a caller cannot pass neither and land in an unguarded branch.
6. **Keep static analysis at zero from the beginning.** Findings accumulate faster than they are cleared, and a non-zero baseline stops anyone noticing the next one.

13.2 Migration to a Server-Backed Deployment

The single most valuable extension would be to move persistence and authentication to a server. The architecture was designed for this, and the change is contained.

Phase	Change	Impact on existing code
1. Database	Model the nine entities as relational tables. Flight, User and Booking become tables; BookingSegment, Passenger and Payment become child tables of Booking; Seat remains derived, now by a query.	None. The interface definitions in types.ts map directly to table definitions.
2. API	Expose the repository operations as REST endpoints, preserving the existing function signatures.	None yet. The endpoints mirror the functions that already exist.
3. Storage adapter	Rewrite lib/storage.ts to call the API instead of localStorage.	One file, 102 lines.
4. Asynchrony	Repository functions become async and return Promises; call sites add await.	Mechanical. The Result type is unchanged, so no error-handling logic changes.
5. Transactions	Wrap createBooking in a SERIALIZABLE transaction with a unique constraint on (flight_id, seat_id) for confirmed bookings.	Closes the residual concurrency window of §11.1.2 completely, and makes a multi-segment write atomic by contract.

Phase	Change	Impact on existing code
6. Authentication	Move password hashing to the server; issue HTTP-only, Secure, SameSite session cookies; enforce authorisation server-side.	Closes the residual authorisation gap of §5.6. The domain rules in auth.ts port directly.
7. Domain layer	No change whatsoever.	pricing.ts, seats.ts, validation.ts and ids.ts are pure functions of their inputs and are indifferent to where the data came from.

Table 13.1 Migration plan to a server-backed deployment.

The final row is the point of the exercise. Four of the domain modules — the fare engine, the seat logic, the validation rules and the identifier generation, comprising the majority of the system's business value — require no change at all. That is what the layering bought.

13.3 Immediate Technical Recommendations

- **Bring a second developer onto the codebase.** The single-developer arrangement described in Section 11.2.1 delivered a coherent system but left no peer review of code and concentrated all knowledge of it in one person. The first act of any continuation should be to have a second developer work through the domain layer against this report until they can change it confidently. The outstanding presentation-layer findings below are the clearest evidence of what the absence of a reviewer costs.
- **Clear the static-analysis findings.** The 18 errors and 10 warnings in Table 9.5 are concentrated in the presentation layer and none is deep.
- **Remove unused dependencies.** An audit found several packages declared in package.json that are not imported anywhere. Each is a maintenance liability and a supply-chain risk for no benefit.
- **Add end-to-end test automation.** The 24 manual system test cases would be well served by a browser automation tool, allowing them to run on every change alongside the unit suite.
- **Add component-level tests.** The domain is thoroughly tested and the interface is not. Tests for the seat map and the booking wizard would close the largest remaining gap.
- **Measure coverage.** The suite is comprehensive by design, but instrumented line and branch coverage would identify any remaining untested path objectively.

13.4 Functional Enhancements

- **Connecting flights with through-fares.** A route-finding algorithm over the schedule graph, with minimum connection times per airport and a fare constructed for the itinerary rather than per leg.
- **Ancillary products.** Checked baggage, meal preferences and priority boarding, priced as separate line items on the existing fare breakdown.

- **Online check-in.** A check-in window before departure, permitting seat changes and issuing a boarding pass.
- **Booking modification.** Changing the date or the passenger on an existing booking, with a fare-difference calculation and a change fee.
- **Loyalty programme.** Points accrual and redemption, and tier-based benefits such as complimentary seat selection.

13.5 Recommendations for Academic Use

- **As a reference implementation.** The layered architecture, the repository pattern and the derived read model are present in a form small enough to read in an afternoon.
- **As a starting point for extension exercises.** The migration plan in Section 13.2 is a well-specified project in its own right, as is any of the functional enhancements above.
- **As a testing case study.** The traceability matrix, the defect log and the deliberate emphasis on negative testing together illustrate a systematic test strategy on a system small enough to comprehend in full.

REFERENCES

Sources are listed alphabetically. Web resources were consulted during the development period.

Beck, K. (2003) Test-Driven Development: By Example. Boston: Addison-Wesley.

Bourque, P. and Fairley, R.E. (eds.) (2014) SWEBOK: Guide to the Software Engineering Body of Knowledge, Version 3.0. Los Alamitos, CA: IEEE Computer Society.

Evans, E. (2003) Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston: Addison-Wesley.

Fowler, M. (2002) Patterns of Enterprise Application Architecture. Boston: Addison-Wesley.

Fowler, M. (2018) Refactoring: Improving the Design of Existing Code. 2nd edn. Boston: Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley.

International Air Transport Association (2023) New Distribution Capability (NDC) Standard. Montreal: IATA.

International Organization for Standardization (2011) ISO/IEC 25010: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE). Geneva: ISO.

Larman, C. (2004) Applying UML and Patterns. 3rd edn. Upper Saddle River, NJ: Prentice Hall.

Martin, R.C. (2017) Clean Architecture: A Craftsman's Guide to Software Structure and Design. Boston: Prentice Hall.

Meta Platforms, Inc. (2024) React Documentation. Available at: <https://react.dev>

Microsoft Corporation (2024) TypeScript Handbook. Available at: <https://www.typescriptlang.org/docs/>

Mozilla Foundation (2024) MDN Web Docs: Web Storage API. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API

Mozilla Foundation (2024) MDN Web Docs: SubtleCrypto. Available at: <https://developer.mozilla.org/en-US/docs/Web/API/SubtleCrypto>

Nielsen, J. (1994) 'Enhancing the explanatory power of usability heuristics', Proceedings of the SIGCHI Conference on Human Factors in Computing Systems. New York: ACM, pp. 152–158.

Open Web Application Security Project (2021) OWASP Top Ten Web Application Security Risks. Available at: <https://owasp.org/www-project-top-ten/>

Pressman, R.S. and Maxim, B.R. (2020) Software Engineering: A Practitioner's Approach. 9th edn. New York: McGraw-Hill.

Sommerville, I. (2016) Software Engineering. 10th edn. Harlow: Pearson Education.

Talluri, K.T. and van Ryzin, G.J. (2004) The Theory and Practice of Revenue Management. New York: Springer.

Vercel, Inc. (2024) Next.js Documentation. Available at: <https://nextjs.org/docs>

World Wide Web Consortium (2023) Web Content Accessibility Guidelines (WCAG) 2.2. Available at: <https://www.w3.org/TR/WCAG22/>

APPENDIX A: INSTALLATION AND USER GUIDE

A.1 Prerequisites

- Node.js version 18 or later.
- npm, which is bundled with Node.js.
- A current web browser: Chrome, Firefox, Edge or Safari.
- No database server, no web server and no network connection are required after installation.

A.2 Installation

```
# 1. Enter the project directory
cd skyroute

# 2. Install dependencies (once)
npm install

# 3. Start the development server
npm run dev

# 4. Open the application
# http://localhost:3000
```

A.3 Available Commands

Command	Purpose
<code>npm run dev</code>	Start the development server with hot reloading.
<code>npm run build</code>	Produce an optimised production build.
<code>npm start</code>	Serve the production build.
<code>npm test</code>	Run the full suite of 211 automated tests once.
<code>npm run test:watch</code>	Re-run affected tests automatically as files change.
<code>npm run test:coverage</code>	Run the suite with coverage instrumentation.
<code>npm run lint</code>	Run ESLint across the codebase.

A.4 Demonstration Accounts

The system seeds itself on first load with 21 days of departures and two accounts. Either may be used immediately; new accounts may also be registered, and a booking may be made with no account at all.

Role	Email address	Password	Access
Customer	customer@skyroute.test	Passw0rd	Search, book, manage and cancel own bookings
Administrator	admin@skyroute.test	Admin@123	All customer functions plus the administration console

A.5 Walkthrough: Making a Booking

1. From the landing page choose a trip type — one way, return or multi-city — then a departure and arrival airport, a date within the next 21 days, a cabin class and the number of passengers, and search.
2. Review the results. Sort and filter as required. Each card shows the per-adult fare and the total for the whole party inclusive of taxes and charges.
3. Select a flight for each leg of the journey.
4. Choose to continue with an account or as a guest.
5. Choose one seat for each fare-paying passenger, on each leg, or use automatic assignment. Infants travel on an adult's lap and are not allocated a seat. Fees for exit-row and window or aisle seats in Economy are shown on the seat itself.
6. Enter each passenger's details. The date of birth must be consistent with the fare type. A passport number is required only on international routes.
7. Choose a payment method. For a card, any number passing the Luhn check is accepted; 4084 0840 8408 4081 is a valid test number. Bank transfer and wallet do not require card details.
8. Review the itemised total for the whole journey and confirm. On success a six-character booking reference is issued and the printable itinerary is displayed.

A.6 Walkthrough: Managing a Booking

1. Signed in, open My Bookings to see all reservations attached to your account, filtered by upcoming, past or cancelled.
2. Alternatively, open Manage Booking and enter a reference together with a passenger surname to retrieve any booking without signing in — this is also how a guest booking is reached.
3. Use the print action to open the printable itinerary; the browser's print dialogue will save it as a PDF.
4. To cancel, the refund is calculated and displayed before the cancellation is committed. Confirm to proceed; seats are released on every leg immediately.

A.7 Troubleshooting

Symptom	Cause and remedy
"Browser storage is unavailable"	The browser is in private or incognito mode, or the storage quota is exhausted. Open the application in a normal window.
No flights appear for any search	The seeded schedule may have expired as real time advanced. Sign in as administrator and use System → Reset to regenerate it.
A booking cannot be found	References are six characters and exclude the letters I and O and the digits 0 and 1. Check for a transcription error, and confirm the surname matches a passenger on the booking.
Data disappeared	Clearing browsing data removes localStorage. There is no backup; this is an inherent property of client-side persistence, discussed in Section 1.6.
Port 3000 is already in use	Run <code>npm run dev -- -p 3001</code> to use a different port.

APPENDIX B: PROJECT STRUCTURE

```

skyroute/
|-- app/                                PRESENTATION - routes
| |-- layout.tsx                        Root layout, providers, navigation
| |-- globals.css                       Design tokens, motion, appearance
| |-- page.tsx                          Landing surface and flight search
| |-- search/page.tsx                  Results, sorting, filtering
| |-- book/[flightId]/page.tsx         Four-step booking wizard
| |-- confirmation/[pnr]/page.tsx      Printable e-ticket
| |-- bookings/page.tsx                My bookings and cancellation
| |-- manage/page.tsx                  Reference + surname lookup
| |-- login/page.tsx                   Sign in
| |-- register/page.tsx                Account registration
| |-- admin/page.tsx                   Administration console (5 modules)
| |-- icon.svg favicon.ico             Favicon, from design/logo-small.svg
| |-- apple-icon.tsx                   Home-screen icon, generated
| +-- opengraph-image.tsx              Social card, generated
|
|-- components/                         PRESENTATION - components
| |-- AppProvider.tsx                  Session context, seeding, tab sync
| |-- Navbar.tsx ui.tsx                Navigation and shared primitives
| |-- Brand.tsx                        The logo mark, two cuts
| |-- icons.tsx                        The 56-icon set
| |-- SearchForm.tsx FlightCard.tsx
| |-- SeatMap.tsx                      Accessible interactive seat map
| |-- PassengerForm.tsx PaymentForm.tsx
| |-- FareSummary.tsx ItineraryCard.tsx AuthShell.tsx
| |-- RouteArc.tsx                     The route that draws itself
| |-- ui/theme-toggle.tsx              Appearance switch
| +-- (landing surface components)
|
|-- lib/                                APPLICATION - DOMAIN - PERSISTENCE
| |-- types.ts                         Domain model (9 entities)
| |-- storage.ts                       <-- ONLY module touching localStorage
| |-- repository.ts                    Use cases, authorisation, transactions
| |-- pricing.ts                       Fare engine, combination, refunds
| |-- seats.ts                         Seat derivation and conflict checks
| |-- validation.ts                    All validation rules, incl. Luhn
| |-- auth.ts                          Salted iterative hashing
| |-- ids.ts                           PNR and identifier generation
| |-- seed.ts                          Airports, routes, aircraft, schedule
| +-- format.ts                        Date, time, currency presentation
|
|-- tests/                              211 AUTOMATED TESTS
| |-- helpers.ts                       Shared fixtures
| |-- validation.test.ts                74 tests
| |-- repository.test.ts                61 tests
| |-- pricing.test.ts                   30 tests
| |-- seats.test.ts                     25 tests
| +-- auth.test.ts                      21 tests
|
|-- design/                             Logo vectors, icon sheet, tooling
+-- package.json tsconfig.json tailwind.config.ts vitest.config.ts

```

APPENDIX C: VERIFICATION EVIDENCE

C.1 Automated Test Suite

```
$ npm test

RUN v2.1.8

OK tests/validation.test.ts (74 tests)
OK tests/repository.test.ts (61 tests)
OK tests/pricing.test.ts (30 tests)
OK tests/seats.test.ts (25 tests)
OK tests/auth.test.ts (21 tests)

Test Files 5 passed (5)
  Tests 211 passed (211)
  Duration 1.45s
```

C.2 Type Check

```
$ npx tsc --noEmit

(no output - zero errors in strict mode)
```

C.3 Static Analysis

```
$ npx eslint . --ext .ts,.tsx

10 errors react-hooks/set-state-in-effect
4 errors @typescript-eslint/no-explicit-any
3 errors react-hooks/purity
1 error react-hooks/use-memo
6 warnings @typescript-eslint/no-unused-vars
4 warnings (exhaustive-deps, aria role, image elements)

x 28 problems (18 errors, 10 warnings)

See Section 9.7. None corresponds to a failing test or observed defect;
all are concentrated in the presentation layer.
```

APPENDIX D: CONTRIBUTION RECORD

The table below records each member's principal area of contribution. Implementation of the software — the architecture, all source code and the automated test suites — was carried out solely by the member listed at S/N 1. The remaining members contributed the requirements analysis, documentation, manual and acceptance testing, and review on which the project equally depends. All members participated in the iteration planning sessions and reviews.

S/N	Name	Matric. No.	Principal contribution
1	Ezedimbu Ebube Emmanuel	2022514081	Team Lead / Sole Developer — Development
2	Uba Thelma Chidinma	2022514029	Assistant Coordinator / Secretary — Coordination & Administration
3	Nnolum Vivian Chisom	2022514008	Scheduling & Meeting Records Officer — Coordination & Administration
4	Micheal Ezra Chukwuebuka	2022514105	Resources & Logistics Officer — Coordination & Administration
5	Anuforo Valentine Uzochukwu	2022514073	Requirements Gathering Lead — Requirements & Research
6	Ogbodo Chimdindu Favour	2022514051	Stakeholder Interview Officer — Requirements & Research
7	Ugwu Anthony Chukwuemeka	2022514076	Comparative Systems Researcher — Requirements & Research
8	Nwachukwu Augustine Ifechukwu	2022514043	Airline Fare & Policy Researcher — Requirements & Research
9	Anekwe Favour Uchechukwu	2022514181	Documentation Lead — Documentation & Writing
10	Unachukwu Chimamanda Jessica	2022514050	Technical Writer — Requirements & Design — Documentation & Writing
11	Ekemezie Chibuikem Azuka	2022514099	Technical Writer — Implementation & Testing — Documentation & Writing
12	Elobisi Michael Chukwuebuka	2022514042	Diagrams & Figures Officer — Documentation & Writing
13	Kenechi Richard Michael	2022514179	User Acceptance Testing Lead — User Testing & Feedback
14	Ogheneakoro Ugochukwu Akpevwe	2022514151	Exploratory Tester — Search & Booking — User Testing & Feedback
15	Okeke Isreal Chukwuka	2022514011	Exploratory Tester — Accounts & Administration

S/N	Name	Matric. No.	Principal contribution
			— User Testing & Feedback
16	Chukwuka Kamsiyochukwu Nelson	2022514185	Defect Log Recorder — User Testing & Feedback
17	Ukohah Onyinye Nkechi	2022514143	Usability & Accessibility Reviewer — Review & Presentation
18	Nwoye David Chiagoziem	2022514172	Editorial & Proofreading Reviewer — Review & Presentation
19	Iheanyi Valentine Kelechi	2022514117	Presentation & Demonstration Lead — Review & Presentation

Signed on behalf of the group:

Signature	Name and matriculation number	Date
	Ezedimbu Ebube Emmanuel (2022514081) — Team Lead / Sole Developer	
	Anekwe Favour Uchechukwu (2022514181) — Documentation Lead	
	Kenechi Richard Michael (2022514179) — User Acceptance Testing Lead	